NOVA

ROCKET PROPULSION · NOZZLE DESIGN

Sean Bowman Isabella Duprey-Churn Compiled 07/20/2026

Nozzle Design Overview

CONTENTS

Document Scope

Nomenclature

Nozzle Design Background

- What is the Method of Characteristics?
- Initial Conditions: Sauer's Transonic Solution
- Initial Conditions: Diverging Section
- Kernel Generation: Creating the Mach Net
- Detailing the Axisymmetric Method of Characteristics Approach
- Final Characteristic: The End of the Mach Net
- Flow Straightening: Putting a Roof on Supersonic Flow

Regenerative Cooling Background

- Ablative Nozzles
- Active Cooling
- What is Regenerative Cooling?

Expander Cycles and Supercritical Coolants

- Aside: Supercritical Fluids

Great, Now Let's Make a Regeneratively Cooled Nozzle

- What are the problems, and what does physics want?
- What options do we have to solve the problems?
- Where do we start?
- Problem Solving: Listening to Physics

Defining the Channel Geometry

- Where does the jacket run?
- Sizing the channel
- Wrapping the channel
- The cross sections
- The jacket
- Modeling the performance
- How far can we trust it?

Design Point

Chamber Conditions

Resulting Geometry and Performance

A Note on Area Ratio

This document is a design reference for regeneratively cooled, shock-free bell nozzles. It covers the physics and the reasoning behind two coupled design problems: generating a supersonic contour via Axisymmetric Method of Characteristics, and designing a regenerative cooling architecture onto that contour.

The two halves are not independent. The contour produced in Part I is the geometric input to Part II: the method-of-characteristics wall becomes the hot wall that the cooling channels are offset from, and the exhaust-side heat transfer correlation is evaluated against the local Mach number and area ratio that the Mach net supplies.

Document Scope

PART	SUBJECT	PRODUCES
I	Supersonic contour generation via Axisymmetric Method of Characteristics	Shock-free wall contour, exit conditions, thrust coefficient
II	Regenerative cooling architecture	Circular, rectangular or helical channel jacket from the injector face to the regen truncation, 1-D heat transfer performance estimate
III	Worked design example	A 100 [kN] LOX/LH2 upper stage carried end to end

Part II takes an oxygen-cooled hybrid at approximately 15 [MPa] chamber pressure as its hardest cooling case, and the worked example in Part III is a 100 [kN] LOX/LH2 upper stage. Neither is a constraint on the methods themselves, which are propellant and scale agnostic.

Nomenclature

Symbols are shared across both parts unless noted.

- A = area
- b = linear intercept

- C_+ = left running characteristic
- C_- = right-running characteristic
- c_P = specific heat at constant pressure
- c^* = characteristic velocity
- c_τ = thrust coefficient
- D, d_H = diameter, hydraulic diameter
- d = channel depth, outward from the wall
- e = channel wall roughness
- f = Darcy friction factor
- H = rib (fin) height
- h = convective heat transfer coefficient
- k = thermal conductivity
- L = channel path length
- L_{frac} = length fraction
- LR = left-running
- M = mach number
- m = slope; fin parameter (Part II)
- MW = molecular weight
- N = number of cooling channels, or of helical starts
- Nu = Nusselt number
- P = static pressure; wetted perimeter (Part II sections)
- Pr = Prandtl number
- $\dot{Q}$ = heat transfer rate
- R = specific gas constant; packing radius (Part II circular channels)
- R_c = throat radius of curvature
- $R_{gas}, R_{wall}, R_{coolant}$ = gas-side, wall, and coolant-side thermal resistance
- Re = Reynolds number
- r = radial position; channel radius (Part II)
- r_c = channel corner radius
- RR = right-running
- T = temperature
- s = spacing between helical passes, measured across the channel
- s_m = arc length along the wall meridian

- t, t_{infill}, t_{rib} = hot wall thickness, minimum rib (infill) thickness, local rib thickness
- v = velocity
- w = channel width, across the wall
- x = axial position

Greek:

- γ = ratio of specific heats
- α = Sauer flow parameter
- ΔL = differential path length between cross sections
- ϵ = peak axial distance of mach 1 ahead of physical throat
- η = trough axial distance of mach 1 behind of physical throat; fin efficiency (Part II)
- μ = mach angle; dynamic viscosity (Bartz correlation)
- ν = prandtl meyer angle
- θ = flow angle; channel wrap angle about the axis (Part II)
- σ = Bartz boundary layer correction term
- τ = thrust
- ϕ = helix angle to the wall meridian

Subscripts:

- $differential$ = between two values
- $local$ = at a point in the flow
- $stagnation$ = in the chamber
- $throat$ = at the nozzle throat (smallest cross sectional area)
- $curvature, inlet/outlet$ = in reference to the incoming or outgoing throat curvature
- $streamline$ = in reference to a flow streamline
- $sonicline$ = values on Sauer's solution
- $sonic$ = velocity when Mach = 1 for a given fluid
- $axial$ = in the direction of the nozzle axis
- $flow$ = with respect to the local flow direction
- $sauer$ = any property that is a derivative of Sauer's solution
- $intersection$ = in reference to a property calculated at a characteristic intersection
- $wall$ = in reference to the known throat wall locations
- $guess$ = approximate value that will be numerically converged

- $max, adiabatic$ = assuming no heat transfer to the surroundings
- u, d = in reference to *upstream* and *downstream* mach net points
- aw = adiabatic wall
- hw, cw = hot wall and cold wall, the gas-side and coolant-side faces of the wall
- 0 = initial/chamber properties
- e = nozzle exit plane properties

Superscripts/Overheads:

- dot ($\dot{a}$) = per time
- bar ($\bar{a}$) = non-dimensional
- star (a^*) = ratio

Part I: Nozzle Contour Design

This part covers the design of converging-diverging nozzle contours via Axisymmetric Method of Characteristics (AxMoC). The contour that comes out of this part is the hot wall that Part II cools.

Nozzle Design Background

From the absolutely massive F-1 engines that lifted the Saturn V, to the smallest Reaction Control thruster that orients a satellite in space, the physics of flow through supersonic nozzles and the fundamental design concepts that describe them are the same!

However, when it comes to the design of nozzles, there are a few different approaches. The design problem typically starts with some mission requirements that nail down important aspects of the design, such as propellant combination and thrust. With the defining parameters in place, the nozzle design may take either a very simple or dramatically complex approach:

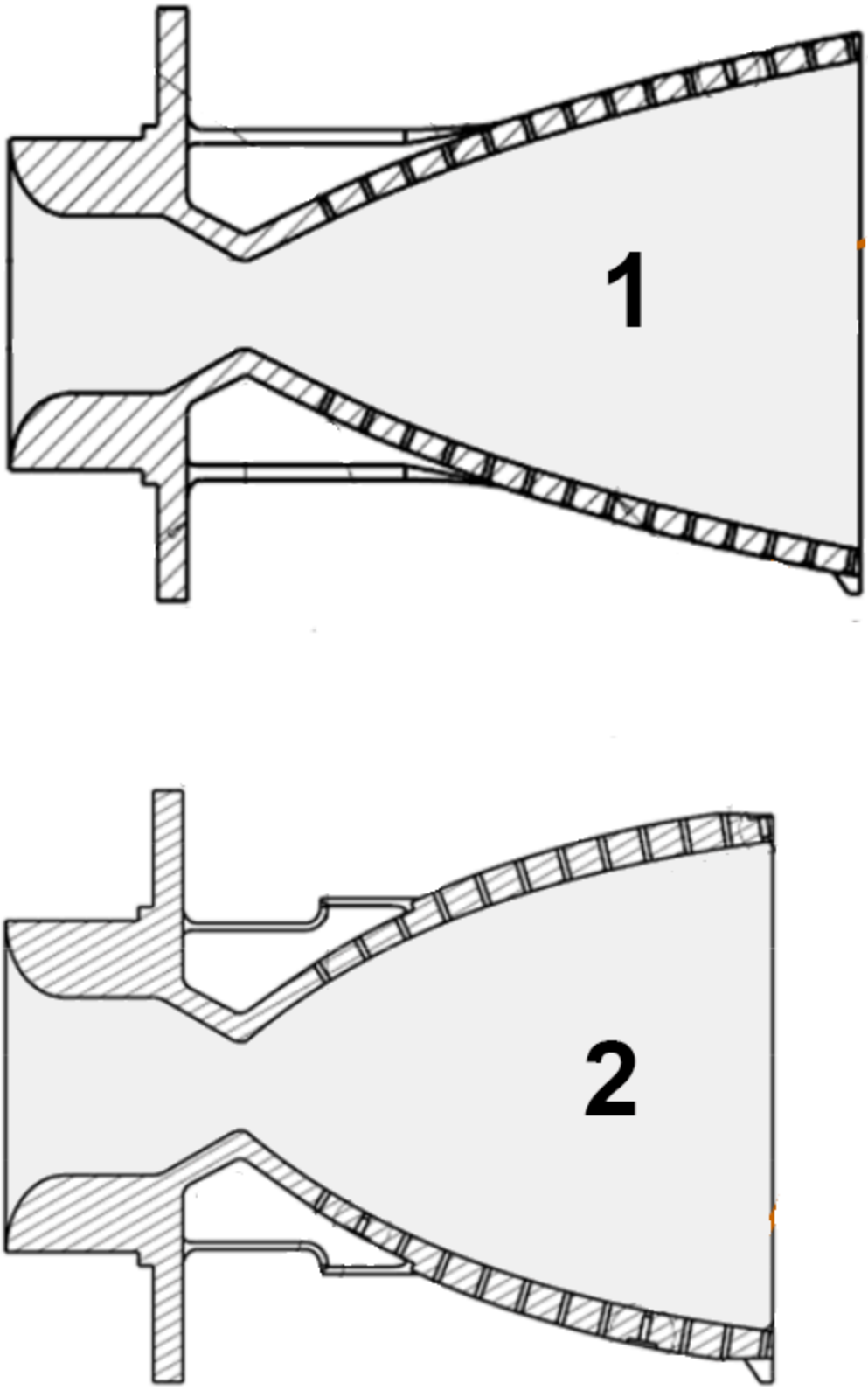
The simple approach is to define the target outlet pressure that the exhaust flow will be expanding to, which is equivalent to specifying what altitude the nozzle is optimized for. Based on the propellants, we can use this information to define an exit mach number, and then specify the area ratio of the nozzle accordingly. This process assumes nothing of the geometry of this nozzle, just the geometric properties at the nozzle boundaries (namely the throat and exit areas and subsequent radii). This is nozzle design in its simplest form, just a list of critical requirement numbers.

However, we have to eventually make the thing so we need to define what shape is going to connect the throat and outlet of our nozzle. Depending on the use case, some designers may fall back on established principles and elect to make manufacturing easy by making the simplest possible nozzle design: a cone!

[Figure: Cone Nozzle Reference Image] Conical Nozzle Reference [7]

For a cone of a 15 degree half angle between the specified throat and exit radii, the specific impulse (I_{sp}) efficiency (or how far off the measured I_{sp} is from isentropically ideal I_{sp}) is near 98% [7]. For the uninitiated, I_{sp} is vaguely comparable to a rocket engine's *fuel efficiency*, similar to miles per gallon (mpg) fuel efficiency for a car. The higher the number (for rockets I_{sp} is generally somewhere around 300[s]), the more force you are able to generate per mass of propellant, and the term *isentropic* just means that the energy conversion to force was perfect. The unit of *seconds* can sometimes be confusing but it represents the duration of thrust production that is possible for a given amount of propellant to produce under its own weight. This 98% conversion rate is well good enough for designers that are only concerned with manufacturability and not inching out every fractional percentage point in performance, and is a common design choice for simple solid rockets and student projects.

However, conical nozzles are the longest a functional and efficient nozzle could be, because the distance traversed from the throat to the exit is linear. This becomes a problem as nozzles get very large or are intending to go to space and therefore need to maximize their weight efficiency. We could reach that target exit area much faster if we traversed the gap between throat and exit radii in a parabolic fashion instead, leading to the natural next step of bell nozzles!



Cross sections of a truncated ideal contour (1) and a parabolic contour (2) for the same throat and exit. Credit: NASA, public domain

But how much shorter than a cone do we want to be? We could make a nozzle that is half the length of a cone, but will the flow still expand as efficiently as just using a cone? (Spoiler alert, no it won't) It is possible to move the wall too far from the flow and end up with another problem: supersonic flow slamming into your wall and bouncing off! This is also known as a shock! So, we want to cut down on length but we don't want to introduce shocks and therefore efficiency losses, so how long do we make a parabolic nozzle? The convention generally is around 75% of the length of an equivalent 15 degree conical nozzle, and this achieves similar performance at a length reduction of ~25%, which saves substantially on overall weight.

This is all fine and good, but it feels very imprecise. We are doing rocket science, after all, isn't there a better way? There sure is! Up until now we have been talking primarily about the desires of designers and ease of manufacturing, but have we stopped to consider what the supersonic flow wants? When you take a physics-first approach to solving the problem things get much more complicated, but the juice is worth the squeeze so-to-speak. What if there was a way to follow the supersonic expanding flow as it did what it naturally wants to, and just trace a line through it for our nozzle wall? That is the philosophy behind nozzle design through a Method of Characteristics approach.

What is the Method of Characteristics?

It's all characteristics? Always has been.

It may be surprising if you have never heard of the term before, but the Method of Characteristics (MoC) has nothing to do with nozzle design specifically. MoC is just a technique for solving partial differential equations that apply to hyperbolic fields and is applicable far outside of its use in nozzle design.

Conveniently for us, however, the supersonic flow field after the throat of a converging-diverging nozzle is described by hyperbolic field equations that we can solve using the MoC approach, allowing us to build a supersonic flowfield after the throat of a nozzle. If we assume that the flow is comprised of an ideal gas that is steady and maintains constant total enthalpy, then we can model the flow using:

- *2nd Law of Thermodynamics (assuming shock-free, inviscid, irrotational flow)*
- *Conservation of Mass*
- *Conservation of Momentum*

$$\vec{\nabla} \times \vec{v} = 0 \tag{1}$$

$$\nabla \cdot \left(\rho \vec{v}\right) = 0 \tag{2}$$

$$\rho \frac{D\vec{v}}{Dt} + \vec{\nabla} P = 0 \tag{3}$$

It can be shown (not here, for the interested reader of the full derivation see **Ferri, A., "The Method of Characteristics," General Theory of High Speed Aerodynamics, edited by W. Sears, The Macmillan Co.,**

1951) that these governing equations can be simplified to form the *four* differential equations we need to solve the flowfield, two *characteristic* equations and two *compatibility* equations (or rather one equation for each of the right and left-running characteristics from any one point in the field):

$$\left(\frac{dr}{dx}\right)_{characteristic} = tan\left(\theta \mp \mu\right) \tag{4}$$

$$0 = \frac{dv}{v} \mp tan(\mu)d\theta - \frac{tan(\mu)sin(\mu)sin(\theta)}{cos\left(\theta \pm \mu\right)}\frac{dx}{r} \tag{5}$$

The *compatibility* equations that we are solving for our 2D Axisymmetric flow case are of the form:

$$d(\theta \mp \nu) = \frac{1}{\sqrt{M^2 - 1} \pm cot(\theta)}\frac{dr}{r} \tag{6}$$

which can be shown to be writable in the form of equation (5) through some heavy manipulation that is present in the referenced paper for the interested reader. Both forms will be of use to us moving forward.

For me it helped to remember that the terms *right-running* and *left-running* are speaking with respect to the nozzle axis, so when looking along the axis the *left-running* characteristic goes to the left and is denoted as C_+ and the *right-running* characteristic goes to the right and is denoted as C_-.

[Figure: MoC Reference Image]

Partial differential equations can sometimes be challenging to solve, and there are simplifying assumptions and techniques that can be employed to solve these equations describing the flowfield. One simplification is to assume that the flow is perfectly planar (or 2D), and in such a case we can reduce the PDEs that describe the flowfield down to simple algebraic expressions that are outright solvable given initial conditions of the flow. This is the common approach explored in an undergrad fluids class, where the implementation is simple and effective at explaining the concept.

We, however, are not quite as lucky. The flowfield is *not* actually 2D, which would be more representative of flat nozzles that are used in supersonic wind tunnels. Our flow is more closely approximated by assuming that the flow is axisymmetric about the nozzle axis, which leads us naturally to bell nozzles. In our case, instead of simple algebra, we are tasked instead with solving the system of Ordinary Differential Equations (ODEs) given by the *characteristic* and *compatibility* equations (4, 5). More on the implementation of the governing equations later.

Once the flowfield is described, we can simply trace a streamline through the flow to draw our desired nozzle contour. Building the flowfield is a process that happens in stages that are defined by what

properties of the flow we are able to assume at each stage. The MoC approach employed here is broken into 3 distinct sections: throat kernel, inner expansion mesh, and flow straightening section.

[Figure: Characteristic Mesh Sections]

The Mach net (or the collection of all mesh points at which we now know the mach number and flow angle) is generated in that order, and each step will be further detailed below. Sounds simple enough (maybe) but there are a few things that we need to pin down before we can actually generate a nozzle contour.

Initial Conditions: Sauer's Transonic Solution

Sauer, what a guy.

In order to propagate flow information downstream of the throat and build out our flowfield, we need to prescribe some boundary conditions. One set of conditions that we can know with confidence are the flow properties at the throat of the nozzle, thanks to the work of Sauer [2]. Sauer and his extensive study of transonic flows at the throat of converging-diverging nozzles gives us a wealth of information that is critical to nozzle design. Using his technique, we are able to characterize the sonic line in the nozzle throat (where a much more appropriate approximation of the transition to Mach 1 occurs) and use the information there as an initial condition for our MoC solution.

[Figure: Sauer Reference Image]

Sauer's solution involves the use of two parameters, α and ϵ (and also $\eta = \epsilon$ for round nozzles), to determine the location of the sonic line at the nozzle throat:

$$\alpha = \sqrt{\frac{2}{(\gamma+1) r_{throat} r_{curvature.inlet}}} \tag{7}$$

$$\eta = \epsilon = \frac{r_t}{8} \sqrt{2(\gamma+1)\frac{r_{throat}}{r_{curvature,inlet}}} \tag{8}$$

where we can build a distribution of points describing the sonic line by:

$$x_{sonicline}(r) = -\frac{\gamma+1}{2}\alpha r^2 + \epsilon \tag{9}$$

$$0 \leq r \leq r_{throat} \tag{10}$$

and we can sample Mach numbers to calculate flow velocity at varying radii by solving:

$$M_{Sauer}\sqrt{\frac{\gamma R T_{stagnation}}{1+\frac{\gamma-1}{2}{M_{Sauer}}^2}} - v_{axial} = 0 \tag{11}$$

$$v_{axial}(r) = v_{sonic}\left(1+\left[\alpha\left(x_{sonicline}(r)-\epsilon\right)+\frac{\gamma+1}{4}(\alpha r)^2\right]\right) \tag{12}$$

$$v_{sonic} = \sqrt{\gamma R T_{sonic}} \tag{13}$$

$$T_{sonic} = \frac{T_{stagnation}}{1+\frac{\gamma-1}{2}} \tag{14}$$

for M_{Sauer} using some numerical zero-ing technique.

The parameter $\epsilon = \eta$ represents the physical distance before and after the throat where the location of Mach 1 actually occurs (see the above reference image), and what I am calling the "flow parameter" α is a constant of proportionality applied to the velocity potential equations when solving the initial flow field described in the x and y directions by:

$$u(x,r) = \alpha x + \frac{\gamma+1}{4}\alpha^2 r + \dots \tag{15}$$

$$v(x,r) = \frac{\gamma+1}{2}\alpha^2 x r + \frac{\gamma+1^2}{16}\alpha^3 r^3 + \dots \tag{16}$$

A reasonable initial guess for that solve is the ratio of the local axial velocity to the sonic velocity, which converges quickly for the full range of throat radii of curvature of interest.

Initial Conditions: Diverging Section

Rao, what another guy.

Another influential individual in the nozzle design space is G.V.R. Rao [3,7], who determined empirical relationships for nozzle throat inlets and outlets through extensive study and experimentation. Today, the Rao throat inlet and outlet are considered synonymous with nozzle design and you will widely see the values for throat curvature cited essentially as fact. It is important to note that there are other ways to generate nozzle throat geometry (one of which involves a MoC approach that propagates backwards and defines the throat contour) but Rao nozzles have been shown to be incredibly efficient so for now that can serve as a boundary condition for our purposes.

The values for Rao's throat curvature are:

$$r_{curvature,inlet} = 1.5 r_{throat} \tag{17}$$

$$r_{curvature,outlet} = 0.382 r_{throat} \approx 0.4 r_{throat} \tag{18}$$

These should look familiar from the reference image between conical and bell nozzles above.

For the diverging section of the nozzle we only need $r_{curvature,outlet}$, however both are useful to define for the generation of the converging section of the nozzle in the future.

By convention, the Rao throat diverging section extends out to a wall angle equal to 1/4 of the Prandtl-Meyer angle at the target exit Mach Number. The Prandtl-Meyer angle of a supersonic flow turning as it passes an expanding section is defined as:

$$\nu = \sqrt{\frac{\gamma+1}{\gamma-1}} \arctan\left(\sqrt{\frac{\gamma-1}{\gamma+1}(M^2-1)}\right) - \arctan\left(\sqrt{M^2-1}\right) \tag{19}$$

With the throat wall and Sauer solution boundary conditions defined, we can visualize the initial conditions of our non-dimensional design space using the relationships from equations (8, 17):

[Figure: Boundary Conditions]

Kernel Generation: Creating the Mach Net

Propagate those characteristics, they do be intersectin'

At every point in the forthcoming Mach net we will be keeping track of the Mach Number and the Flow Angle at that location (as well as the (x, r) coordinates themselves). With our boundary conditions defined, we can calculate the first point of our Mach net by calculating where Sauer's solution intersects our throat wall. This is accomplished by solving:

$$\tan\left(\frac{\theta_{flow} + \mu_{Mach=1} + \mu_{Sauer}}{2}\right)\left(\epsilon - x_{throat} - \frac{\gamma+1}{8}\alpha {r_{intersection}}^2\right) + r_{throat} - r_{intersection}$$

$$\mu_{Sauer} = \arcsin\left(\frac{1}{M_{Sa}}\right.$$

for $r_{intersection}$ again using some zero-ing method and, as a result:

$$x_{intersection} = \epsilon - \frac{\gamma+1}{8}\alpha r_{intersection}^2 \tag{22}$$

$$M_{intersection} = M_{Sauer} \tag{23}$$

$$\theta_{intersection} = \theta_{wall} \tag{24}$$

Here we already have a strategy from the previous section to calculate M_{sauer} for equation (20).

The parameter μ, the Mach angle, describes the conical angle of potential information propagation in a supersonic flow downstream of a supersonic object.

[Figure: Mach Angle]

The characteristic intersection follows directly: solve equation (19) for $r_{intersection}$, then evaluate equation (21) for the axial location.

[Figure: Initial Conditions]

The first step in generating the throat kernel is complete, but where do we go from here? One of our assumptions is that the flow angle at the throat wall is the same as the wall angle, and we know all of the wall angles by definition. To that end, we have x, r, θ_{flow} at all throat wall points and can project a characteristic point off of the wall given that we know the "upstream" and "current" points of the mesh from the wall. We can make an initial guess for the characteristic projection point by averaging the flow angles at nearby nodes and checking whether or not the guessed point satisfies the compatibility equations near the wall. This can be done numerically through a residual minimizer, where the flow angle and mach number at the intersection point are the control variables of the minimization:

$$
\begin{aligned}
0 =& \frac{1}{\sqrt{M^2-1} \mp \cot\left(\theta_{flow,intersection}\right)} * \left(\frac{r_{intersection} - r_{1,2}}{r_{intersection}} - \left(\theta_{flow,intersection} \pm \nu_{interse} \right. \right. \\
res_{C_-} =& \frac{1}{\sqrt{M_{guess}^2-1} - \cot\left(\theta_{flow,guess}\right)} * \left(\frac{r_{intersection} - r_1}{r_{intersection}} - \left(\theta_{flow,guess} + \nu_{guess}\right) - \left(\theta_1 + \right. \right. \\
res_{C_+} =& \frac{1}{\sqrt{M_{guess}^2-1} + \cot\left(\theta_{flow,guess}\right)} * \left(\frac{r_{intersection} - r_2}{r_{intersection}} - \left(\theta_{flow,guess} - \nu_{guess}\right) - \left(\theta_2 - \right. \right. \\
res =& \sqrt{res_{C_-}^2 + res_{C_+}^2}
\end{aligned}
$$

where,

$$
\begin{aligned}
x_{intersection} =& \frac{b_{RR} - b_{LR}}{m_{LR} - m_{RR}} && (29) \\
r_{intersection} =& \frac{m_{LR} x_{intersection} + b_{LR} + m_{RR} x_{intersection} + b_{RR}}{2} && (30) \\
m_{L,R} =& \tan\left(\theta_{flow} + \mu_{guess}\right), \tan\left(\theta_{flow,average} - \mu_{average}\right) && (31) \\
b_{L,R} =& r_{1,2} - m_{L,R} x_{1,2} && (32)
\end{aligned}
$$

The general form of equations should be recognizable from the compatibility equations presented in equation (6).

The residual is minimized over the two control variables M and θ_{flow} at the intersection, starting from M_1 and the average of the two neighboring flow angles.

Here at the first step of the kernel generation, we do not have any additional internal points to calculate that would necessitate the use of Axisymmetric Method of Characteristics (AxMoC), so just for this step we will skip over that and return to it in a moment when it is relevant.

We also need to calculate the location where the projected characteristic will intersect the limiting characteristic, given the last point in the characteristic mesh before reaching the limiting characteristic. That can be done by checking, via convergence, where the projecting characteristic satisfies Sauer's solution:

$$-\frac{\gamma+1}{8}\alpha r^2+\epsilon-\left(\frac{r_{intersection}-\left[r_{throat}-x_{throat}\tan\left(\mu_{Sauer}\right)\right]}{\tan\left(\mu_{Sauer}\right)}\right)=0 \tag{33}$$

$$x_{intersection}=\epsilon-\frac{\gamma+1}{8}\alpha r_{intersection}^2 \tag{34}$$

As before we have a procedure for calculating where a characteristic intersects the limiting characteristic, which we can follow again to figure out where this intersection occurs.

We can visualize the first characteristic projection and first limiting characteristic intersection like this:

[Figure: First Characteristic Projection and Limiting Characteristic Intersection]

Detailing the Axisymmetric Method of Characteristics Approach

With the first points of the mesh defined, we can perform our first actual step of Axisymmetric Method of Characteristics (AxMoC) toward the throat wall to find the next throat wall intersection point that will be the seed to the next step of AxMoC.

To do this, we must provide the algorithm with information about two "upstream" points in the flow and use the information at those points to propagate flow information "forward" in the mesh. The first time we do this can be a check to make sure everything is going smoothly, by choosing the throat intersection (white) and the limiting characteristic intersection (green) as our first trial points and recalculating the wall characteristic projection point (red) but this time with AxMoC. This is relevant because the compatibility equations from before are of the same form as the flowfield equations, just solved near the wall.

We will need the $x_i, r_i, M_i, \theta_{flow,i}$ (we can shorthandedly call this package of information about both seed points the AxMoC *kernel*) for each point we are choosing to provide the AxMoC algorithm. From there we can calculate some dependent parameters:

$$\nu_i = arcsin\left(\frac{1}{M_i}\right) \tag{35}$$

$$T_{local,i} = \frac{T_{stagnation}}{1 + \frac{\gamma-1}{2}M_i^2} \tag{36}$$

$$v_{local,i} = M_i\sqrt{\gamma R T_{local,i}} \tag{37}$$

$$\bar{v}_{local,i} = \frac{cot(\nu_i)}{v_{local,i}} \tag{38}$$

$$LRT = \frac{sin(\theta_{flow,1})sin(\nu_1)}{sin(\theta_{flow,1} + \nu_1)} \tag{39}$$

$$RRT = \frac{sin(\theta_{flow,2})sin(\nu_2)}{sin(\theta_{flow,2} - \nu_2)} \tag{40}$$

Where LRT and RRT are shorthand expressions for the *left-hand term* and *right-hand term* for the respective characteristics (and it makes it easier to code to break the variables apart).

With the input points and dependent properties defined, we can now set up the solution process for the system of ODEs that will be solved to obtain our characteristic intersection point and its resultant properties $x_{intersection}, r_{intersection}, M_{intersection}, \theta_{flow,intersection}$. This process will require guessing and checking at the actual intersection point and properties, as the underlying expressions are not algebraic but differential equations, so each time these properties are calculated the variance in the calculated intersection point is checked until it falls within a specified tolerance. The *compatibility* ODEs that we are solving for our 2D Axisymmetric flow case are of the form:

$$d(\theta \mp \nu) = \frac{1}{\sqrt{M^2 - 1} \pm cot(\theta)}\frac{dr}{r} \tag{41}$$

as before, which can be shown to be writable in the form of equations (5) from earlier (presented here again as a reminder of general form so the expressions below feel motivated by the underlying ODEs):

$$0 = \frac{dv}{v} \mp tan(\mu)d\theta - \frac{tan(\mu)sin(\mu)sin(\theta)}{cos\big(\theta \pm \mu\big)}\frac{dx}{r} \tag{42}$$

which is equivalent to:

$$0 = cot(\mu)\frac{dv}{v} \mp d\theta - \frac{sin(\mu)sin(\theta)}{cos\big(\theta \pm \mu\big)}\frac{dx}{r} \tag{43}$$

when dividing out by $tan(\mu)$.

For a small enough distance between mach net points (or rather a sufficient enough resolution in the numerical space), we can assume that the traversal between points is approximately linear, despite the fact that the characteristics in reality are curved. We can discretize the compatibility equations for a first-order numerical scheme and calculate the approximate local characteristic slopes assuming a linear relationship between mesh points. This allows us to back out the intersection point downstream in the characteristic mesh:

$$m_{LR} = tan(\theta_{flow,1} + \mu_1) \tag{44}$$
$$m_{RR} = tan(\theta_{flow,2} - \mu_2) \tag{45}$$
$$x_{intersection} = \frac{r_2 - r_1 - x_2 m_{RR} + x_1 m_{LR}}{m_{LR} - m_{RR}} \tag{46}$$
$$r_{intersection} = r_1 + (x_{intersection} - x_1) m_{LR} \tag{47}$$

We can calculate the velocity and flow angle at the intersection point as:

$$v_{intersection} = \frac{v_1 \bar{v}_1 + v_2 \bar{v}_2 + \frac{LRT}{r_1}(r_{intersection} - r_1) + \frac{RRT}{r_2}(r_{intersection} - r_2) + \theta_{flow,1} - \theta}{\bar{v}_1 + \bar{v}_2}$$
$$\theta_{flow,intersection} = \theta_{flow,1} + \bar{v}_1(v_{intersection} - v_1) - \frac{LRT}{r_1}(r_{intersection} - r_1) + \theta_{flow,2} + \bar{v}_2(v_{inte}$$

The form of these equations comes from discretizing and then solving the compatibility equations for each respective variable.

This set of equations breaks down near the nozzle axis where r is near 0, so instead when the intersection point is near the axis we must solve for the intersections in a different manner by solving the system all at once using its matrix representation:

$$Ax = B \tag{50}$$

$$\epsilon = \frac{v_1 \bar{v}_1 + v_2 \bar{v}_2 + \frac{RRT}{r_2}(r_{intersection} - r_2) + \theta_{flow,2}}{\bar{v}_1 + \bar{v}_2} \tag{51}$$
$$A = \begin{bmatrix} 1 & \frac{-\bar{v}_1}{2} & \frac{-\bar{v}_1 v_1}{2} \\ -\frac{1}{\bar{v}_1 + \bar{v}_2} & 1 & \epsilon \end{bmatrix} \tag{52}$$
$$B = rref(A) = \begin{bmatrix} 1 & 0 & v_{intersection} \\ 0 & 1 & \theta_{flow,intersection} \end{bmatrix} \tag{53}$$

Where the matrix B is the *reduced row echelon form* of the matrix A, the last column of which contains the solutions for $v_{intersection}$ and $\theta_{flow,intersection}$.

Finally, the mach number and mach angle at the intersection point are calculated as:

$$v_{ratio} = \frac{v_{intersection}}{v_{max,adiabatic}} \tag{54}$$

$$M_{intersection} = \sqrt{\frac{2}{\gamma - 1}\left(\frac{v_{ratio}}{1 - v_{ratio}}\right)^2} \tag{55}$$

$$\mu = arcsin\left(\frac{1}{M_{intersection}}\right) \tag{56}$$

This is a good first approximation of the characteristic intersection point, however we can do much better by creating a second order solution by iterating upon the new intersection point and using it to average flow properties between the upstream points and the newly calculated intersection point and recalculating a new characteristic intersection. We can repeatedly check how much the newly calculated intersection point has changed since the last iteration and stop the convergence once the value of the intersection point change falls within a specified tolerance of 1e-8. This allows is to numerically converge to a downstream characteristic intersection with a high degree of confidence for any two starting points in the mesh.

Each pass therefore reduces to: approximate the characteristic slopes and their intersection, equations (43-46); solve for velocity and flow angle at that intersection, using the matrix form of equations (49-52) when the point falls on the axis and equations (47, 48) otherwise; and recover the mach and mach angles from equations (54, 55). The kernel is then re-averaged against the new intersection point and the pass repeats until the intersection location settles.

The intersection point between the kernel points for the first pass of AxMoC should directly overlap with the characteristic projection from the previous step, since that projected point came from solving the compatibility equations near the wall.

[Figure: Axisymmetric Method of Characteristics Initial Corrector Step]

Looks like we nailed it!

For right now the fact that the method is *axisymmetric* doesn't seem to matter, but it will feel more relevant later during the generation of the mach net near the nozzle axis.

The next phase of the process involves a similar procedure but for finding the location where a characteristic intersects the nozzle throat wall. We have a non-dimensional description of the location of the throat wall from assuming incoming and outgoing throat radius of curvature, so we can use that information in conjunction with linear approximations for the characteristic to determine locations of throat wall intersections. We can determine the axial location of the potential throat wall intersection by numerically solving:

$$-\sqrt{(r_{curvature,outlet}r_{throat})^2 - x_{intersection}{}^2} + r_{throat} + r_{curvature,outlet}r_{throat} - m_{LR}x_{intersection}$$

Now, we can propagate a characteristic out until we intersect the throat wall, at which point we can calculate a new characteristic projection and the cycle repeats.

[Figure: Axisymmetric Method of Characteristics Next Step]

This process can then be repeated back and forth between the limiting characteristic and the throat wall until we run out of throat wall points to intersect with. At that point, we have reached the end of the throat kernel.

[Figure: Axisymmetric Method of Characteristics Next Step]

[Figure: Axisymmetric Method of Characteristics Next Step]

[Figure: Axisymmetric Method of Characteristics Next Step]

[Figure: Axisymmetric Method of Characteristics Next Step]

[Figure: Axisymmetric Method of Characteristics Next Step]

Now that we have defined the throat kernel, we can move on to the rest of the mach net. We can start by projecting a right-running characteristic down to the nozzle axis to act as the boundary point to perform method of characteristics out until we run out of throat kernel reference points. To do this we can start in the throat kernel and perform the AxMoC algorithm process but for an entire characteristic, instead of the intersection between two characteristics. The code logic is very similar, however now we are repeating the process across n points between the throat kernel and the nozzle axis. The process flow looks like:

The same treatment applied across n points, rather than to a single intersection, walks a complete characteristic out from a seed point. This allows us to calculate any characteristic line (both *right-running* and *left-running*) originating from a mesh point, of which we want the right-running (C_-) characteristic originating in the throat kernel until it intersects the nozzle axis. This will now act in the same way that the Sauer Limiting Characteristic did as a boundary for generating characteristic intersections.

[Figure: Initial Right Running Characteristic]

[Figure: Inner Expansion Mesh Step]

[Figure: Inner Expansion Mesh Step]

[Figure: Inner Expansion Mesh Step]

Once we reach the nozzle axis, finally we are able to utilize the assumption that the flow is *axisymmetric* by copying a kernel point across the axis to create the seed points for AxMoC that we need.

[Figure: Inner Expansion Mesh Step]

From there we can repeat the AxMoC process until we converge the kernel down to the nozzle axis.

[Figure: Inner Expansion Mesh Step]

[Figure: Inner Expansion Mesh Step]

[Figure: Inner Expansion Mesh Step]

Final Characteristic: The End of the Mach Net

Starting at the final point of the inner expansion mesh (the dark blue one in the above plot), we can define the final characteristic using the Prandtl-Meyer angle at the calculated exit mach number at the last characteristic mesh point. We can then project a long straight characteristic at that angle off of the final inner expansion mesh point knowing that the flow angle along this characteristic is essentially 0 at every point on the characteristic and the mach number is the calculated exit mach number from the final point of the inner expansion mesh. That straight characteristic ends at the calculated exit radius required for the calculated exit mach to exist at the nozzle wall.

[Figure: Final Characteristic]

Using these points as the new seed points for AxMoC, we can fill out the remainder of the mach net in the flow straightening section, stopping the propagation of points after roughly approximating the distance needed to fully encapsulate the flow field where a wall will be. Theoretically we can propagate this flow information far beyond where we want to bound the flow with a wall, but doing so would be computationally inefficient.

[Figure: Axisymmetric Method of Characteristics From Last Characteristic]

[Figure: Axisymmetric Method of Characteristics Flow Straightening Mesh]

We have now built a representation of a supersonic expanding flowfield where we know the mach number and flow angle at all points. We could conceivably continue the propagation of information further down the nozzle axis, but a restriction has been placed on the design space to terminate calculation if a non-dimensional radius of 1.5 is exceeded before reaching the end of the mach net. This ensures that the nozzle is not larger and heavier than the equivalent conical nozzle, and fits into the larger design problem of optimizing the system in terms of thrust coefficient (which we will discuss further later).

Flow Straightening: Putting a Roof on Supersonic Flow

The last step is to trace a streamline through the flow, starting at the end of the throat wall until we reach the end of our mach net. This streamline will act as our nozzle wall. If we have done a good job, then this nozzle contour should be a representation of the supersonic expanding flow and not impede the flow in any way, which results in a shock-free, isentropic nozzle. The methodology here is a loose amalgomation of three concepts: approximating streamline curvature via bilinear interpolation of mach net points, streamline/characteristic intersections, and flow continuity via streamline tangency. Deep breath, we'll go through each thing slowly.

If we represent a streamline inside of a small region of the characteristic mesh as a quadratic curve (which is the simplest polynomial expression that can represent curvature) then we can say that a streamline can be represented as:

$$r_{streamline}(x) = c_0 + c_1(x - x_u) + c_2(x - x_u)^2 \tag{60}$$

where the centering term $(x - x_u)$ locates the streamline in the flow relative to the mach net points of interest.

If we assume that the streamline originates from the upstream mach net point then:

$$r(x_u) = r_u \tag{61}$$
$$c_0 = r_u \tag{62}$$

The next condition we can utilize is flow tangency along a streamline:

$$\left(\frac{dr}{dx}\right)_u = tan(\theta_{flow,u}) \tag{63}$$
$$c_1 = tan(\theta_{flow,u}) \tag{64}$$

Moreover, we can extract more information by determining flow properties at the intersection between a characteristic ($C_{+/-}$) and the streamline. For the breakdown below the derivation will be done with respect to the left-running characteristic, but the approach is analagous for the right-running characteristic.

$$\left(\frac{dr}{dx}\right)_{intersection} = c_1 + 2c_2(x_{intersection} - x_u) = tan(\theta_{flow,intersection}) \tag{65}$$

From here on out, to make the nomenclature simpler, we will refer to $x_{intersection}$ as $x_{query,LR}$ because this is representative of a potential wall point on a flow streamline that is calculated with respect to the left-running characteristic. There are two solutions for $x_{query,LR}$ due to the quadratic nature of the

streamline, and subsequently there are two more with respect to the right-running characteristic, but more on that later. We can approximate the flow angle and radial location at the $query, LR$ point by linearly interpolating between the downstream and left-running mach net points:

$$\theta_{flow,query,LR} = \theta_{flow,d} + (\theta_{flow,LR} - \theta_{flow,d})\frac{x_{query,LR} - x_d}{x_{LR} - x_d} \tag{66}$$

$$r_{query,LR} = r_d + (r_{LR} - r_d)\frac{x_{query,LR} - x_d}{x_{LR} - x_d} \tag{67}$$

Plugging in terms to the original quadratic expression and solving for c_2 leaves us with:

$$c_2 = \frac{\big(tan(\theta_{flow,LR}) - tan(\theta_{flow,d})\big)(x_{query,LR} - x_d)}{2(x_{query,LR} - x_u)(x_{LR} - x_d)} + \frac{tan(\theta_{flow,d}) - tan(\theta_{flow,u})}{2(x_{query,LR} - x_u)} \tag{68}$$

Now, we can set the expression for $r_{query,LR}$ equal to the original streamline equation and collect terms:

$$r_{query,LR} = r_u + \tan(\theta_{flow,u})(x_{query,LR} - x_u) + c_2(x_{query,LR} - x_u) \tag{69}$$

Substituting the expression for c_2:

$$r_{query,LR} = r_u + \tan(\theta_{flow,u})(x_{query,LR} - x_u) + \left[\frac{\tan(\theta_{flow,LR}) - \tan(\theta_{flow,d})}{2(x_{LR} - x_d)}(x_{query,LR} - x_d) + \frac{\tan(\theta_{flow,d}) - \tan(\theta_{flow,u})}{2}\right](x_{query,LR} - x_u$$

From the condition that the $r_{query,LR}$ can be approximated by linear interpolation:

$$r_u + \tan(\theta_{flow,u})(x_{query,LR} - x_u) + \left[\frac{\tan(\theta_{flow,LR}) - \tan(\theta_{flow,d})}{2(x_{LR} - x_d)}(x_{query,LR} - x_d)(x_{query,LR} - x_u) + \frac{\tan(\theta_{flow,d}) - \tan(\theta_{flow,u})}{2}\right.$$
$$= r_d + \frac{r_{LR} - r_d}{x_{LR} - x_d}(x_{query,LR} - x_d)$$

Multiply both sides by $(x_{LR} - x_d)$ to simplify fractions:

$$(x_{LR} - x_d)r_u + (x_{LR} - x_d)\tan(\theta_{flow,u})(x_{query,LR} - x_u) + \frac{\tan(\theta_{flow,LR}) - \tan(\theta_{flow,d})}{2}(x_{que}$$
$$+ \frac{\tan(\theta_{flow,d}) - \tan(\theta_{flow,u})}{2}(x_{query,LR} - x_u)(x_{LR} - x_d) = (x_{LR} - x_d)r_d + (r_{LR} - r_d)(x_{qu}$$

Expanding $(x_{query,LR} - x_d)(x_{query,LR} - x_u)$:

$$(x_{query,LR} - x_d)(x_{query,LR} - x_u) = x_{query,LR}^2 - x_{query,LR}x_u - x_{query,LR}x_d + x_u x_d \tag{73}$$

After combining like terms, we arrive at a quadratic equation of the form:

$$x_{query,LR}^2 - \eta_{1,LR}x_{query,LR} + \eta_{2,LR} = 0 \tag{74}$$

Where:

$$\eta_{1,LR} = (x_u + x_d) + \frac{2(r_{LR} - r_d) - (x_{LR} - x_d)\big(\tan(\theta_{flow,u}) + \tan(\theta_{flow,d})\big)}{\tan(\theta_{flow,LR}) - \tan(\theta_{flow,d})}$$

$$\eta_{2,LR} = (x_u x_d) + \frac{2(x_{LR} - x_d)(r_u - r_d) + 2(r_{LR} - r_d)x_d - x_u(x_{LR} - x_d)\big(\tan(\theta_{flow,u}) + \text{ta}}{\tan(\theta_{flow,LR}) - \tan(\theta_{flow,d})}$$

We can now solve for the streamline as:

$$r_{streamline}(x) = x^2 - \eta_{1,LR}x + \eta_{2,LR} \tag{77}$$

That was a lot, and ultimately it's just here for completeness because this derivation does not seem to exist in its entirety in any one reference and not ever once in the original code implementation, so I feel better now that it lives here.

In practice, to find valid streamline solutions we need the (x, r, M, θ) information about the mach net at 4 different points: upstream, downstream, and points from the left and right-running characteristics. There are two solutions found relative to the left-running characteristic, and two more for the right-running characteristic, only *one* of the total four is a valid streamline point that will fall axially between the upstream and downstream mach net points . We start by calculating them all and verifying which point falls within the bounding boxes between each seed point. To calculate each wall query point we can solve for the zeros of the quadratic expression:

$$x_{query,LR/RR} = \frac{\eta_{1,LR/RR} \pm \sqrt{\eta_{1,LR/RR}^2 - 4\eta_{2,LR/RR}}}{2} \tag{78}$$

$$r_{query,LR/RR} = r_d + \frac{(r_{LR/RR} - r_d)(x_{query,LR/RR} - x_d)}{x_{LR/RR} - x_d} \tag{79}$$

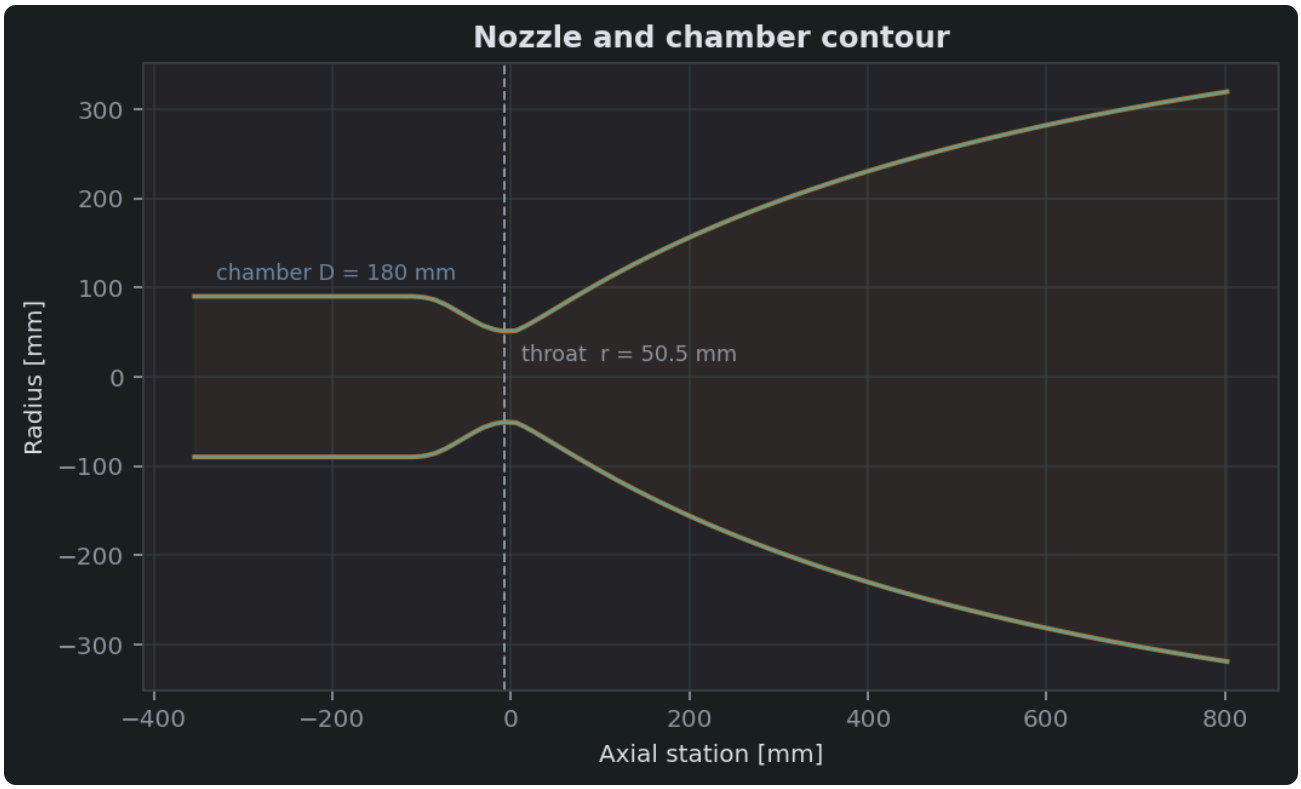
The four candidate points follow from equations (74, 75) and (77, 78); the one that falls between the upstream and downstream mach net points is the wall point, and it becomes the upstream point for the next query. If no candidate qualifies, the end of the mach net has been reached.

With the four wall query points calculated, we can check whether they fall within the bounding boxes:

[Figure: Axisymmetric Method of Characteristics Wall Point Query]

Here the only valid solution is the green point, so we select it as a wall point and repeat the process using the wall point as the next "upstream" point.

[Figure: Axisymmetric Method of Characteristics Wall Point Query]



Generated truncated ideal contour behind its generated chamber, with the chamber diameter and the sized throat radius called out. The shipped engine jackets the whole wall; with the jacket cut short, the figure draws the cooled portion apart from the extension. LOX/LH2 100 [kN] upper stage example case

Now that we have calculated the set of wall points, what remains is to cut the wall and work out what the nozzle delivers. This is only the first candidate contour, because a truncated ideal contour has two design numbers, an area ratio and a length, and one free parameter: the exit mach number the underlying ideal nozzle is designed to. A larger design mach number opens the wall faster near the throat, so it reaches a given area ratio in less length and leaves a steeper exit; that is the trade the outer solve works with.

The wall is cut where it reaches the requested area ratio, which is the truncation criterion NASA SP-8120 attributes to Ahlberg. The design mach number is then swept until the length falling out of that cut is the requested fraction of a 15 degree conical nozzle of the same area ratio. Both design numbers are satisfied by the same solve, and the exit pressure is a result rather than a target: the exit plane of a truncated contour is strongly non-uniform, its wall carries the highest static pressure and lowest mach number on the plane, and a design with two numbers fixed has no free parameter left with which to also

fix a third. If a length fraction is not specified, the truncation length is swept as well, with delivered thrust coefficient as the objective.

The initial values that provide a representative initial condition are calculated using empirical 0-D approximations. Those empirical values for the ideal exit mach number and nozzle truncation length fraction are:

$$M_{ideal} = \sqrt{\frac{2}{\gamma - 1}\left[\left(\frac{P_o}{P_{e,target}}\right)^{\frac{\gamma-1}{\gamma}} - 1\right]} \tag{80}$$

$$L_{frac} \approx 0.8 \tag{81}$$

Ordinarily the thrust coefficient can be approximated as:

$$c_\tau = \frac{\tau}{A_t P_o} \tag{82}$$

where τ is the thrust, A_t is the throat area of the nozzle, and P_o is the chamber pressure. This method provides us with the theoretical ideal thrust coefficient for any nozzle configuration, which can be roughly thought of as how effectively a nozzle converts the chamber pressure into thrust.

We are able to be much more precise about the calculation of thrust coefficient because we have information about the flow angle at all positions along the nozzle exit plane thanks to our mach net. We can use that information to discern what percentage of the flow is pointed along the nozzle axis and therefore contributes to the usable thrust. The rest of the flow that is pointed off of the nozzle axis is wasted, and decreases the calculated thrust coefficient. The problem of nozzle design now centers around creating a nozzle that satisfies both a sufficient thrust coefficient (where most of the flow points along the nozzle axis) while being small and short enough to be length and weight efficient. All nozzles generated with this method will be shock-free in theory, so the game revolves largely around matching target exit mach number with an appropriate nozzle length.

We can calculate the actual thrust coefficient as:

$$c_\tau = c_{\tau,v} + c_{\tau,P} \tag{83}$$

$$c_{\tau,v} = \left[\sqrt{\frac{2\gamma^2}{\gamma-1}\frac{2}{(\gamma+1)^{\gamma-1}(\gamma-1)}\left(1 - \left(\frac{P_{e,node}}{P_o}\right)^{\frac{\gamma-1}{\gamma}}\right)}\right] cos(\theta_{flow,node}) A^*_{differential} \tag{84}$$

$$c_{\tau,P} = \frac{(P_{e,node} - P_a)A_{differential}}{P_o A_t} \tag{85}$$

Where $P_{e,node}$ is the average pressure between two exit plane nodes, $\theta_{flow,node}$ is the average flow angle between exit plane nodes, and $A^*_{differential} = \frac{A_{differential}}{A_e}$ is the area ratio of the annular area between

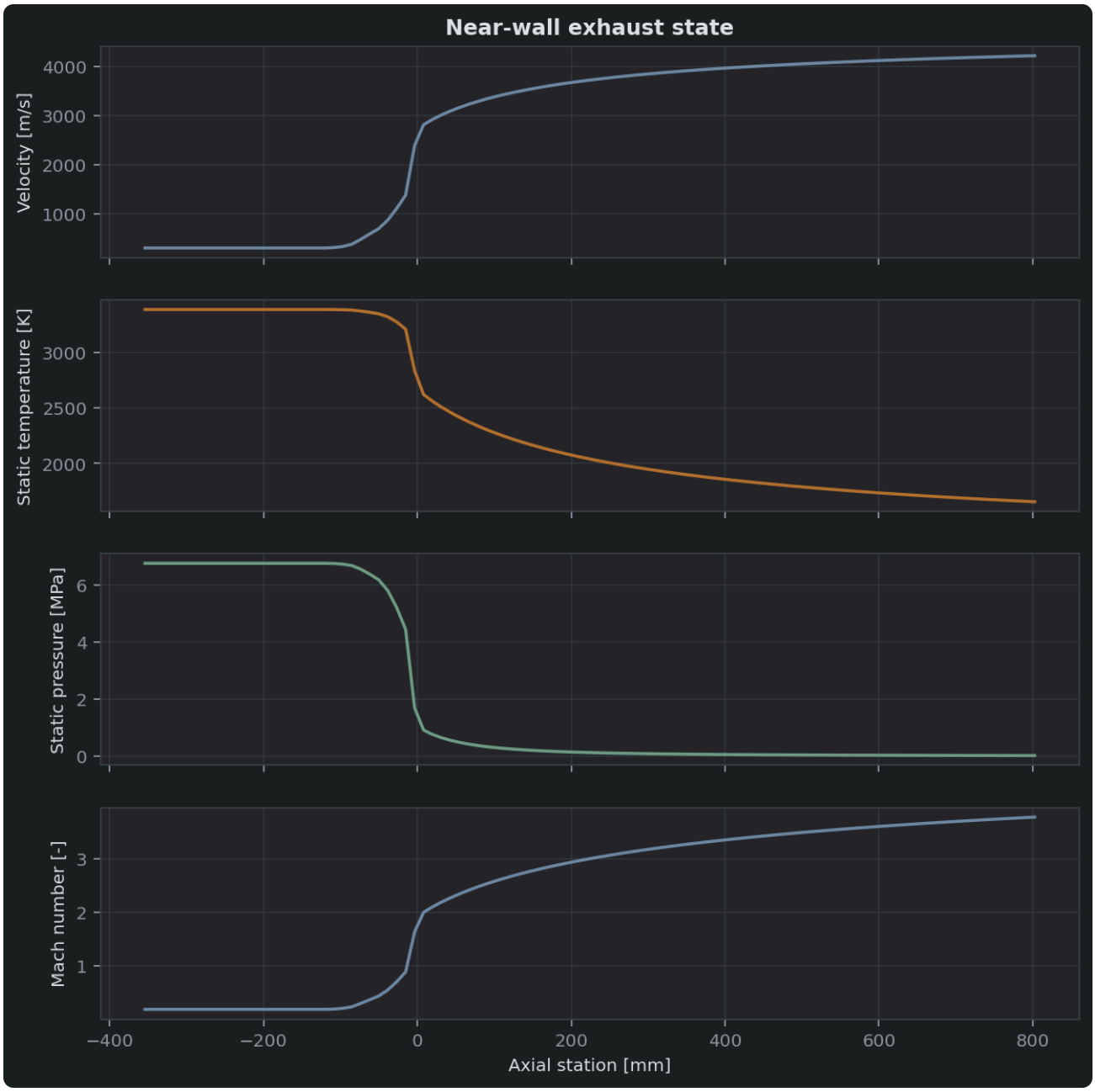
nodes and the nozzle exit area. We can see here that as the flow angle approaches zero, the calculation of thrust coefficient approaches the lightweight approximation of thrust coefficient assuming perfectly straight flow. The total thrust coefficient is the sum of all small annular contributions to thrust coefficient as we integrate along the nozzle exit plane:

$$c_{T,total} = \sum_{i}^{n} \left(c_{T,v,i} + c_{T,P,i} \right) \tag{86}$$

[Figure: Axisymmetric Method of Characteristics Initial Corrector Step]

Part II: Regenerative Cooling Architecture Design

This part covers the design of the regenerative cooling architecture applied to the contour from Part I and the combustion chamber upstream of it. Together they are treated here as the hot wall: channel centerlines are parallel offsets of it, and the exhaust-side heat transfer coefficient is evaluated using the local Mach number and area ratio along it.



Near-wall exhaust velocity, Mach number, static temperature, and static pressure along the nozzle axis. These distributions are the exhaust-side boundary condition for the heat transfer model in this part. LOX/LH2 100 [kN] upper stage example case

Regenerative Cooling Background

One of the fundamental challenges when working with rocket hardware is designing around the extreme physics conditions that rocket components are subjected to. Combustion chambers can reach temperatures comparable to the surface of the sun, while cryogenic propellants can be kept at temperatures near absolute zero (the coldest anything in the universe could possibly be!). During

operation, rocket nozzles that are directing the super-hot exhaust flow need to maintain their shape to continue to operate as designed, but the temperature of most exhaust flows far exceeds the melting temperatures of possible materials that nozzles could be made of.



RS-25 engine 2063 during a hot fire on the Fred Haise Test Stand at Stennis Space Center. Credit: NASA/Chris Russell, public domain

Faced with a physics-based challenge, there are two primary approaches to the problem of nozzles eroding during operation.

Ablative Nozzles

Let it melt!

The first approach is commonly referred to as "ablative" nozzle design, where the nozzle material is allowed to ablate but the rate of ablation is known and the motor/engine operation can be tailored to that ablation rate. This approach is often seen for small duration burn motors (such as missiles or small student rockets) or solid propellant applications where there is not a readily available cooling fluid.

The challenge with ablation is that the size and shape of the nozzle contour changes over time, which changes the fundamental properties of the nozzle (most notably the size of the nozzle throat, which controls the total mass flow rate and subsequent thrust production). Many ablative nozzle designs are

only ablative near the nozzle throat where the highest heat transfer occurs, while the rest of the nozzle contour is made of some well-characterized high-temperature material such as a phenolic resin.

[Figure: TRL Nozzle]

TRL ablative nozzle pre- and post-fire

[Figure: Thrust vs Time]

Thrust output decrease over burn duration

Active Cooling

Fighting the Sun

Allowing performance to vary with time can often be impractical for orbital applications if it is avoidable, making ablative nozzles disadvantageous for space launch vehicles or reusable applications. The alternative approach to nozzle design involves maintaining the nozzle contour shape throughout the entire burn duration (and for any number of conceivable burns), which requires some form of active cooling of (or heat transfer reduction to) the nozzle hot wall. This allows nozzles to maintain their designed performance throughout an entire long-duration burn of multiple minutes or be reused indefinitely.

[Figure: Same Nozzle]

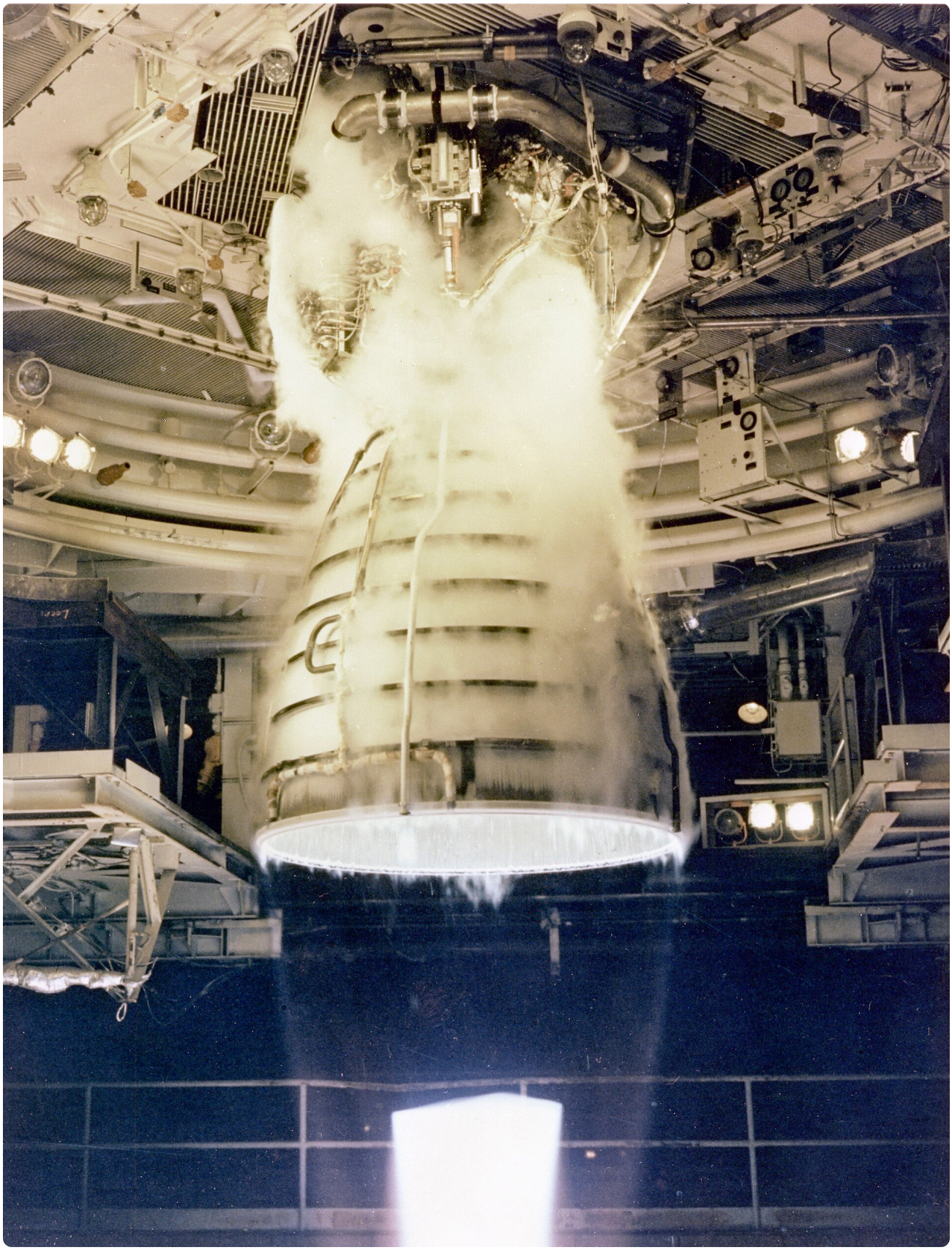
Same nozzles, same performance, different stage of mission

But how can we keep something so hot from melting? In liquid bi-propellant and hybrid rocket engines, there is often a supply of cryogenic liquid that is being stored as propellant to feed the engines that can be utilized as a cooling fluid during operation. This has many advantages beyond the obvious one of maintaining nozzle geometry. The reason propellants are stored in their cryogenic liquid form is for volumetric efficiency, because carrying the equivalent mass of gaseous propellant would require huge high-pressure tanks and be very impractical. In their liquid form, common propellants can be hundreds of times more dense than their gaseous counterparts and therefore be storable in a much smaller volume. However, the propellant must be turned into a gas as some point before the combustion process, because only gaseous materials are capable of combusting. We're in need of a heat exchanger to expand the liquid into a gas before it reaches the combustion chamber, and there just happens to be a very convenient heat exchanger at the end of the engine directing the exhaust flow (the nozzle!).

[Figure: Heat exchanger diagram]

Generic heat exchanger

In liquid bi-propellant systems the common approach is to utilize the liquid fuel, as it often has a much higher heat capacity than the oxidizer that is also stored on-board. Heat capacity can be thought of as the amount of energy it takes to raise the temperature of a given fluid, so a higher heat capacity means that it takes more energy to raise the temperature of the fluid, or conversely that a cooling fluid can accept much more energy without raising its temperature that much. This is an advantage for fuels as coolants, because there is often a lot of nozzle and chamber wall that needs to be cooled and a limited amount of cooling fluid to work with, making this the more efficient choice.



Space Shuttle Main Engine at full power during a 290 [s] test firing. The RS-25 is regeneratively cooled with liquid hydrogen fuel. Credit: NASA, public domain

However, in hybrid engines the fuel material is a solid, leaving only cryogenic oxidizer as the lone choice for designers of active cooling systems for hybrid engines. This is not great news, as common oxidizers have a substantially lower heat capacity than common fuels.

[Figure: Cp Comparison]

Liquid Methane has about twice the heat capacity of liquid oxygen

Don't panic, though, because there are still things that can be done to successfully cool a nozzle using the oxidizer instead! We will touch more on this later, but with modern design and manufacturing techniques we can get creative with cooling geometry in new and exciting ways to unlock previously unattainable design solutions.

In general, a system that utilizes the available propellant to cool the nozzle walls *and* heats that propellant with energy from the engine operation to drive the engine cycle is called *regeneratively cooled*.

What is *Regenerative* Cooling?

The term *regenerative* refers to the fact that energy from the exhaust flow is recaptured and used to heat the same propellant that will eventually be used in combustion to perpetuate the engine cycle. The propellant acts as a nozzle coolant during operation by being passed between the walls of the nozzle, which allows for the double use of the propellant both in combustion and as a cooling fluid.



A copper nozzle with its axial cooling channels machined in, before the outer jacket is closed out. Credit: Steve Jurvetson, CC BY 2.0

There are other forms of active cooling, such as *film cooling*, where a thin sheet of unburnt gaseous propellant (again, typically the fuel) is passed along the nozzle hot wall to create a boundary between the exhaust and the nozzle wall, which is famously the cooling method used on the nozzle extension of the F-1 engines that lift the Saturn-V!



F-1 engine static firing. The dark band at the nozzle exit, ahead of the orange plume, is the unburnt film coolant. Credit: NASA MSFC, public domain

The dark band before the bright orange of the plume is the unburnt gas film leaving the nozzle, which eventually reacts with the oxygen in the air and burns where the plume turns all orange!

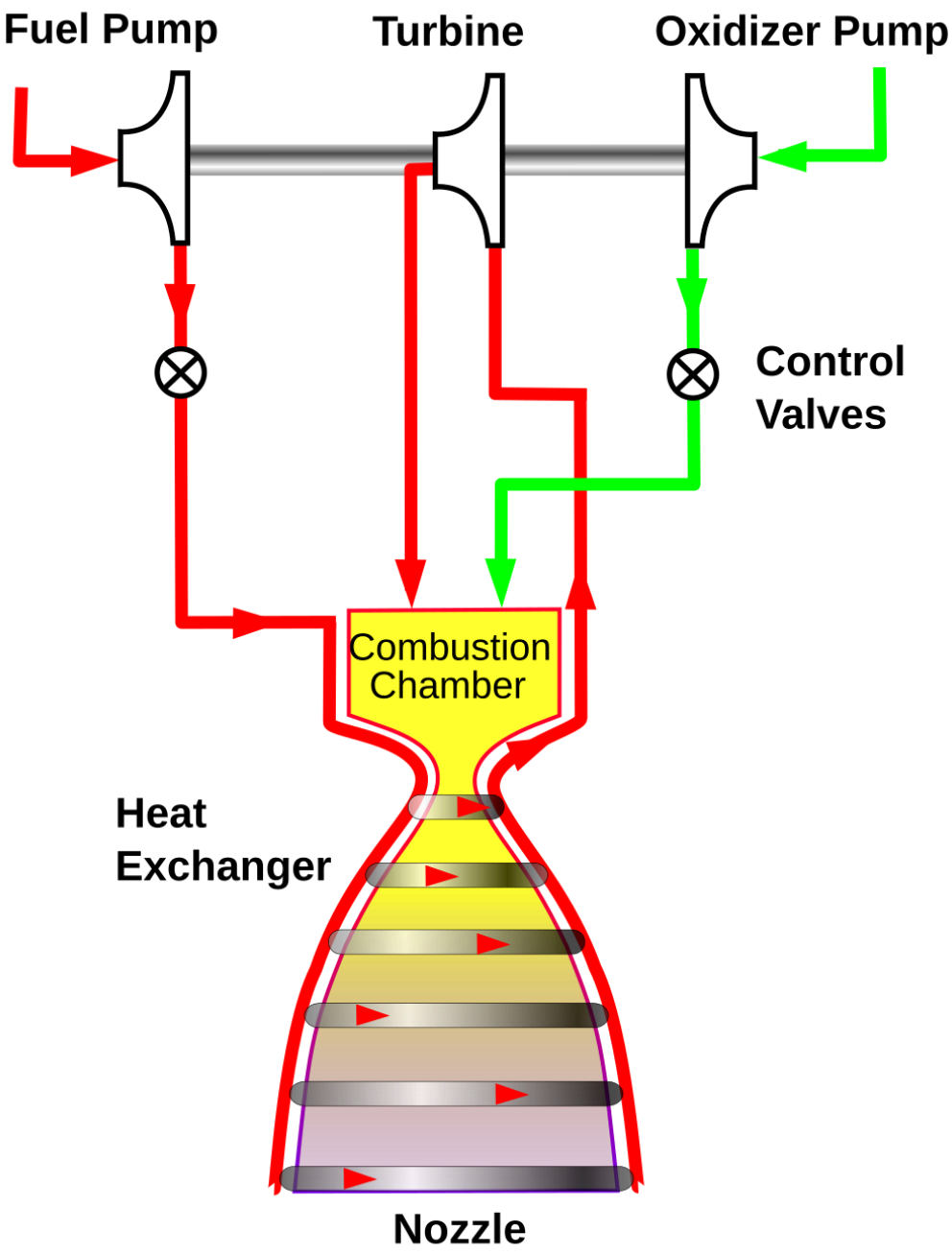
A film on a hybrid nozzle means carrying a separate coolant for it, since a hybrid has no gaseous fuel on hand. NOVA models a film where one exists: `filmCooling.py` lowers the temperature the wall is driven by over the length the film survives, rather than removing heat the way a jacket does. Switch it on with `filmCooling` and give it a coolant, a flow, a temperature, a slot position and a slot height. Two closures are available through `filmCoolingModel`. The Hatch and Papell correlation from NASA TN D-130 is the default and the only one that states its own accuracy, but it was fitted in a constant-area duct and cannot see acceleration or flow turning. The entrainment model of NASA SP-8124 Appendix A accounts for both, through an empirical multiplier read off a design chart, so it is calibrated rather than validated. On a hydrogen film the two disagree by hundreds of kelvin, which is a fair measure of how well film cooling is known at rocket conditions.

The third method is to remove the coolant entirely and let the wall radiate. Past some area ratio the flux has fallen far enough that an uncooled shell settles below its own temperature limit, and carrying a jacket further costs mass and pressure drop for nothing. `radiativeCooling.py` solves that balance along the extension beyond the jacket and reports the margin against the material limit, which is the only question worth asking about an uncooled shell. Switch it on with `makeRadiativeExtension`, and note that it needs a non-`none` `regenTruncationType`, because with none the jacket runs the whole contour and there is no extension to solve.

Both combine with a jacket rather than replacing it, which is why they are described here alongside it rather than as alternatives to it.

Expander Cycles and Supercritical Coolants

The regenerative jacket matters most in the expander cycle, where the heat the coolant picks up in the jacket is what drives the turbine.



Expander cycle flow schematic. Propellant is pumped through the cooling jacket, where it picks up heat from the chamber and nozzle, and the resulting high-energy fluid drives the turbine. Credit: Duk, CC BY-SA 3.0

The expander cycle operates on the premise that a single fluid (typically the fuel) can be *expanded* from its compressed liquid form to a more useable high-energy form (gas/supercritical fluid) to then drive a turbine which perpetuates the engine cycle. A hybrid runs the same cycle on its oxidizer, since its solid fuel cannot flow through a jacket.

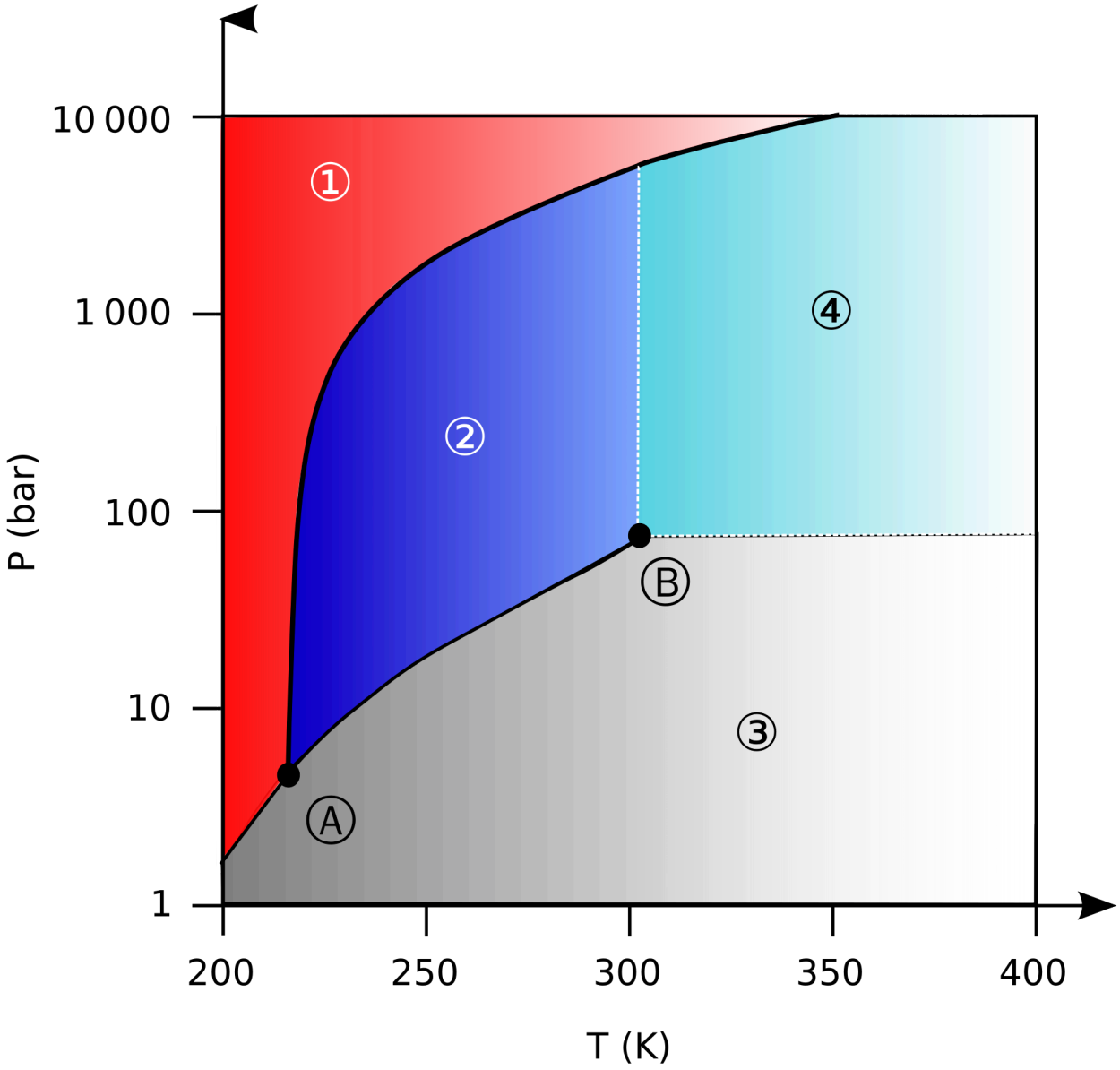
[Figure: Hybrid Expander Cycle]

Hybrid Expander Cycle

Oxygen as the working fluid is not unheard of, but the heat capacity discussed above makes it a secondary choice wherever a liquid fuel is on hand; with a solid-fuel hybrid there is no alternative, and one must make do with what is available. What every expander shares is a high system pressure. Orbital-class engines run their chambers at around 15 MPa (~2000 psi), well above the critical pressures of oxygen (5.04 MPa), methane (4.60 MPa) and hydrogen (1.30 MPa), and fluids past their *critical* state behave differently than fluids at more familiar temperatures and pressures. The configuration in `assets/NOVANozzle.json`, for one, cools with hydrogen entering the jacket at 12 MPa and 30 K, supercritical from the start. The following sections briefly go over the physics of supercritical fluids and how that distinction impacts the design of a rocket fluid system.

Aside: Supercritical Fluids

To be *painfully* clear about the distinction being made here, let's start with the simplest example. At standard temperatures and pressures, referring to a substance as a "fluid" encapsulates the concepts of both "liquids" and "gasses", which share the property that the individual particles that comprise them are able to flow and take the shape of their containers. Liquids are held together by stronger intermolecular forces which causes local fluid density to be high, whereas gasses consist of individual particles whose motion and internal energy state exceed any intermolecular forces, allowing particles to move about independently of one another leading to low local fluid density. At the boundary between these two states of matter there exists a clear delineation in fluid properties and crossing over this boundary causes the state of the fluid to vary dramatically. This is more colloquially known as "vaporization", or more distinctly "evaporation" when the process happens at the liquid surface boundary and "boiling" when the process takes place within the bulk liquid. Something very interesting happens, however, when you increase the pressure past a certain threshold known as the "critical pressure".



Pressure-temperature phase diagram for carbon dioxide, with the supercritical region labeled. Oxygen behaves the same way qualitatively, at its own critical point of 5.04 [MPa] and 154.6 [K]. Credit: CC0 1.0

In the phase diagram above, the critical point represents the physical state of the fluid where the distinction between liquid and gas loses practical meaning. This means that properties of the fluid will no longer change rapidly with increases of temperature or pressure, and instead changes in properties will be gradual and the magnitudes of the properties will be some average of the liquid and gas states. We can visualize this by plotting thermophysical properties (like density and viscosity) instead of fluid phase, to get an idea for how the properties vary with changes in temperature and pressure.

[Figure: Oxygen Properties]

Oxygen Properties

On the left we can see how Oxygen's density varies over the same temperature and pressure ranges from the phase diagram above. There is a steep drop off in density where the phase change from liquid to gas occurs, causing the density to drop from the high liquid value (~1000 [kg/m^3]) to the low gaseous value (~10 [kg/m^3]). However, at higher pressures, the drop off becomes more gradual until eventually there is no drop at all. Instead, the density varies smoothly as the temperature increases (I like to call this region the "density slide").

What this means for rocket engines is that propellant that begins the engine cycle journey as a liquid in the propellant tank does not phase change in the traditional sense at any point in the engine. Instead, we can avoid the rapid property changes and take advantage of the physics of supercritical fluids by operating at a higher system pressure, and we can create more accurate representative physics models by capturing these unique behaviors of supercritical fluids.

[Figure: Oxygen Properties 2]

Oxygen Properties in Engine

Great, Now Let's Make a Regeneratively Cooled Nozzle

To tie this all together, let's apply this knowledge to the design of a regeneratively cooled nozzle from first principles. So far we have not been speaking about legacy design practices, but rather the fundamental physics of the problem, which is a core tenant of the design philosophy used throughout. When you take a first-principles approach, there is often something natural hiding in the physics of the problem that can be taken advantage of. If you are thoughtful enough, you may find it easier to swim with the river rather than against it.

With that in mind we have a physics problem, so let's lay that out and begin to chip away at it.

What are the problems, and what does physics want?

Let's list out some of the fundamental physics challenges that we are facing that are relevant to the design of regenerative cooling architecture:

- Limited coolant capacity (oxygen in a hybrid worst of all)
- Extreme temperature gradient from exhaust to wall and coolant
- Extreme pressure gradient from internal to external volumes
- Achieving adequate propellant conditioning to drive engine cycle
- Minimization of pressure losses in system
- Limited propellant mass flow per time to use as coolant

- Optimum material selection and manufacturability

Some of these things are not changeable and we must simply work around them, such as the amount of mass flow and type of coolant that is on-hand. They are critical to identify, though, because they inform us going forward. Others have more design freedom, and I will try to motivate each thought process and decision made moving forward as we discuss potential solutions to these issues. Broadly speaking, we will focus on the underlying physics to motivate the solutions rather than any existing design practices, however when the comparison to a historical context reveals something interesting I will relate the old way to a new way of thinking. That being said, technological improvements in recent years have unlocked new avenues for designing and manufacturing components that can and will change the way we think about the solutions to these issues.

Unencumbered by legacy design choices, how might we attack these issues from the ground-up? First, let's take inventory of the tools available to us.

What options do we have to solve the problems?

We can leverage two critical concepts to get creative with the problem solving process:

1. Additive Manufacturing
2. Algorithmic Modeling

The first concept, "*additive manufacturing*", is a manufacturing type that materializes components from the addition of material (otherwise known as *3D printing*), which allows for the design of components with complex internal geometries that would be challenging/impossible to subtractively machine. The second concept, "*algorithmic modeling*" is a design approach relying on the creation of a *base set of rules* that designs are forced to follow, and allowing resultant components to emerge naturally from specifying design parameters and allowing some algorithm to determine the resulting geometry. This approach results in something interesting that I have been referencing: here the physics of the problem create the geometry rather than a designers whims. So long as the physics is modeled accurately, such designs will need very little verification of their performance between the models and real life, because their very shape was born of the desires of physics itself.

Such an algorithmic design approach could conceivably be built to accommodate additive manufacturing constraints, which further tightens the rules that the geometry-making algorithm(s) live within. This approach has the added benefit, though, of being able to churn out thousands of parametrized designs to find optimal performance in a challenging design space.

The algorithm(s) need to know about the physics of the problem in its most accurate representation possible to then use that information to create the best performing design, regardless of what those physics conditions are. This is a powerful toolset, because you are not designing just one specific nozzle, but every conceivable nozzle that can have its physics modeled accurately at varying conditions and

scales. This is incredibly helpful and time efficient when the system being designed is largely novel and tweaks to the system are common and numerous. Building tools that build tools has an up front cost, but the juice is worth the squeeze when many of the questions about a design are unanswered.

Where do we start?

With our tools in hand, we can tackle some of the easier problems first. Some things only have a few solutions, after all, and some things are beyond our control entirely (like system requirements, the divine word). For example, the coolant fluid mass flow is simply something we must live with, as it is a fundamental parameter of the overall engine and we cannot change that without changing the engine concept entirely. We must be aware of the value, however, because it will be important information to later feed to an algorithmic design tool to provide insight into the physics of our problem, but more on that later.

What about the material selection, or how to work around the extreme physics conditions? Let's think about what would be ideal and go from there:

Problem Solving: Material Selection

We need to keep the nozzle cool and transfer as much heat as possible into the coolant to drive the engine cycle, so we want something with a high thermal conductivity that can effectively transfer heat, but also something that is strong enough to withstand high internal pressures and other erroneous forces. If we wanted to just resist the heat we could use something with a very low thermal conductivity (like a phenolic resin or ceramic), but in this case we are prioritizing heat transfer to condition propellant, not just keeping the nozzle alive.

Copper is a common material that has a high thermal conductivity, but there's a problem. Copper, as far as metal is concerned, might as well be butter. That just won't do if we expect to hold an extreme pressure inside of the fluid system and survive potential loads from shocks or vibrations. Well, we could use some sort of copper alloy that is much stronger, but we will have to pay a price in the heat transfer because we will have to remove some copper and replaced it with things that are much stronger but have a much lower thermal conductivity. A recent-ish development in copper alloys comes from NASA's Glenn Research Center, named GR-Cop42, which swaps the pure copper out for a ratio of [94% Copper, 4% Chromium, 2% Niobium]. With these ratios the yield and ultimate strength of the material effectively doubles, however you pay a small fee of ~15-20% in reduced thermal conductivity. [6] This trade is very much worth it, making GR-Cop42 a great choice for the cooled portion of the nozzle.

To keep the discussion brief, we have entirely neglected other printable metals that simply don't even make it out of the gate from a heat transfer perspective. However, it is the onus of the designer to properly trade study the material of any given design.

Problem Solving: Extreme Physics Conditions with a Limited Coolant

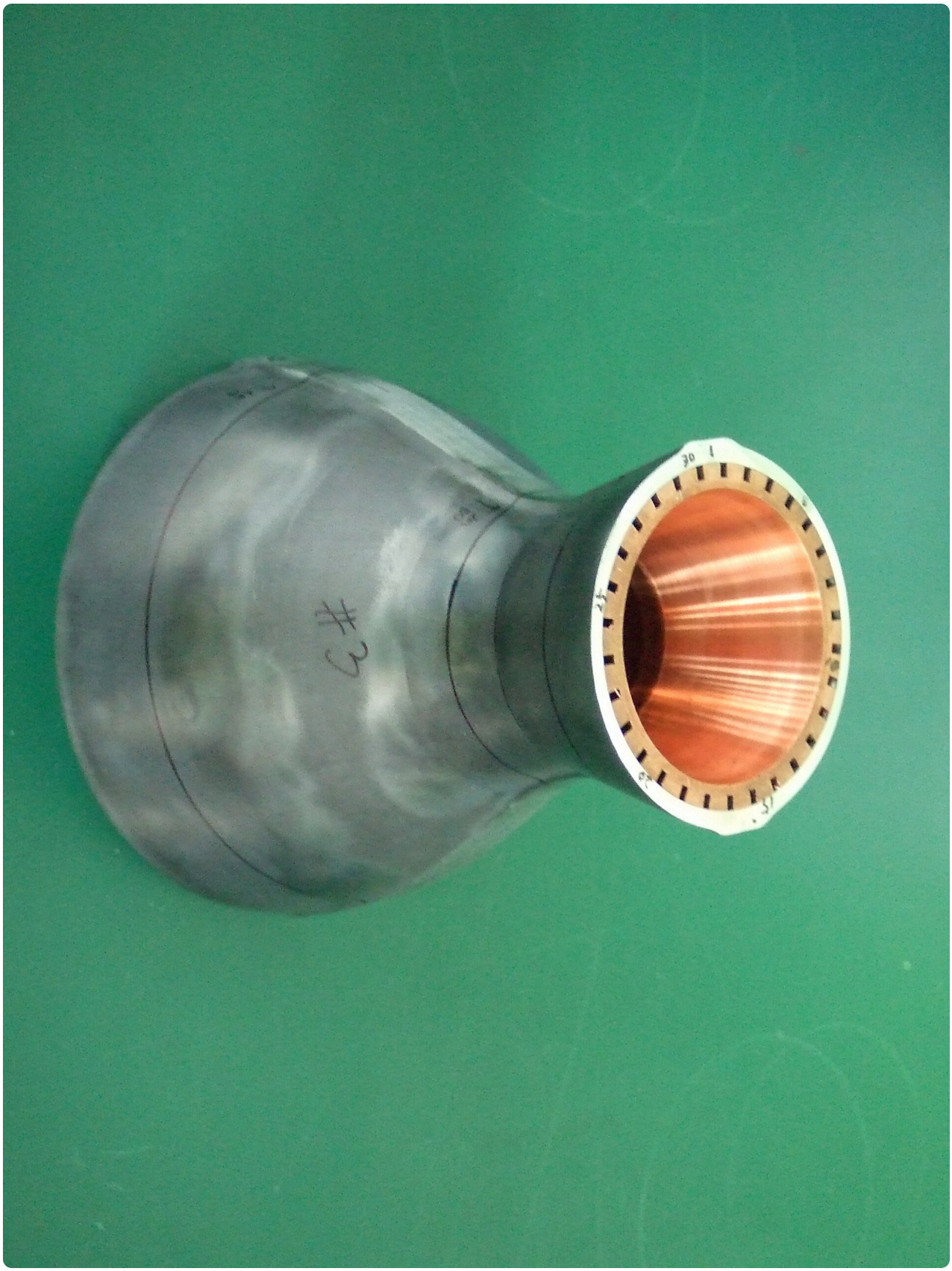
The hardest version of this problem is a hybrid, where oxygen is the only fluid on board that can cool the nozzle, and oxygen has been used as a coolant in lab settings rather than in flight systems. Small engines have it hard too, which at first might sound backwards, but there is a sneaky problem that must be contended with:

In a regeneratively cooled nozzle, the volume of a cooling jacket increases cubically (to the 3rd power) while the nozzle surface area that needs to be cooled increases quadratically (to the second power). This means that as nozzles grow, the conventional wisdom is that the volume of propellant cooling the wall grows faster than the wall that needs to be cooled, so the ability to condition (heat) a propellant to drive an engine cycle gets harder the larger a nozzle becomes (Fans of pop-science may be familiar with the square-cube law problem). This problem happens in reverse for small nozzles, where lower coolant flowrates (due to smaller engines) are responsible for cooling more wall, which is further complicated by using Oxygen as a coolant.

In the days of old there were two common approaches to manufacturing the cooling architecture that were inevitable based on manufacturing constraints: brazed tubes and milled rectilinear channels. In the former case, long circular tubes were bent into the shape of a nozzle contour and welded together, and in the latter case rectangular slots were milled along the outside of a nozzle block and later capped with some sleeve to hold the fluid in.



Looking up the nozzle of an F-1 engine on a Saturn V first stage; the brazed tube-wall cooling channels converge toward the throat. Credit: Clemens Vasters, CC BY 2.0



A chamber assembly viewed end-on: the copper liner sits inside a steel jacket, with the machined axial cooling channels exposed at the flange. Credit: Romanusas2, CC BY-SA 4.0

The geometry of cooling architectures were largely fixed at that point, which was fine for those cases because the fuel material was sufficient to cool the walls and drive the engine cycle.

With a generous coolant and a large engine we may have been able to just place some straight tubes from one end of the nozzle to the other and call it done, but a small engine, or one cooled by oxygen, would be left with one melty nozzle. We need a way to both increase the amount of heat transfer into the coolant, and increase the amount of time it is able to perform heat transfer.

There are competing problems at play that we must contend with: heat transfer and pressure drop. The longer and smaller we make the channels the greater the heat transfer, but the larger the pressure drop. This is because fluid moving faster through a smaller channel has a larger heat transfer coefficient, but also experiences more viscous pressure losses. That pressure is needed downstream to spin a turbine and maintain chamber pressure, so we can't be selfish and eat it all while cooling the nozzle down. We need a way to determine, ideally quickly and at a glance, how small variations in cooling geometry impact the heat transfer and pressure drop performance.

Problem Solving: Listening to Physics

Every lever we have on the coolant side comes down to two things: how fast the coolant moves past the wall, which is set by the channel's flow area, and how much surface it moves past, which is set by the channel's shape. Faster coolant draws more heat and costs pressure. More surface draws more heat too, and it is the cheaper of the two in pressure.

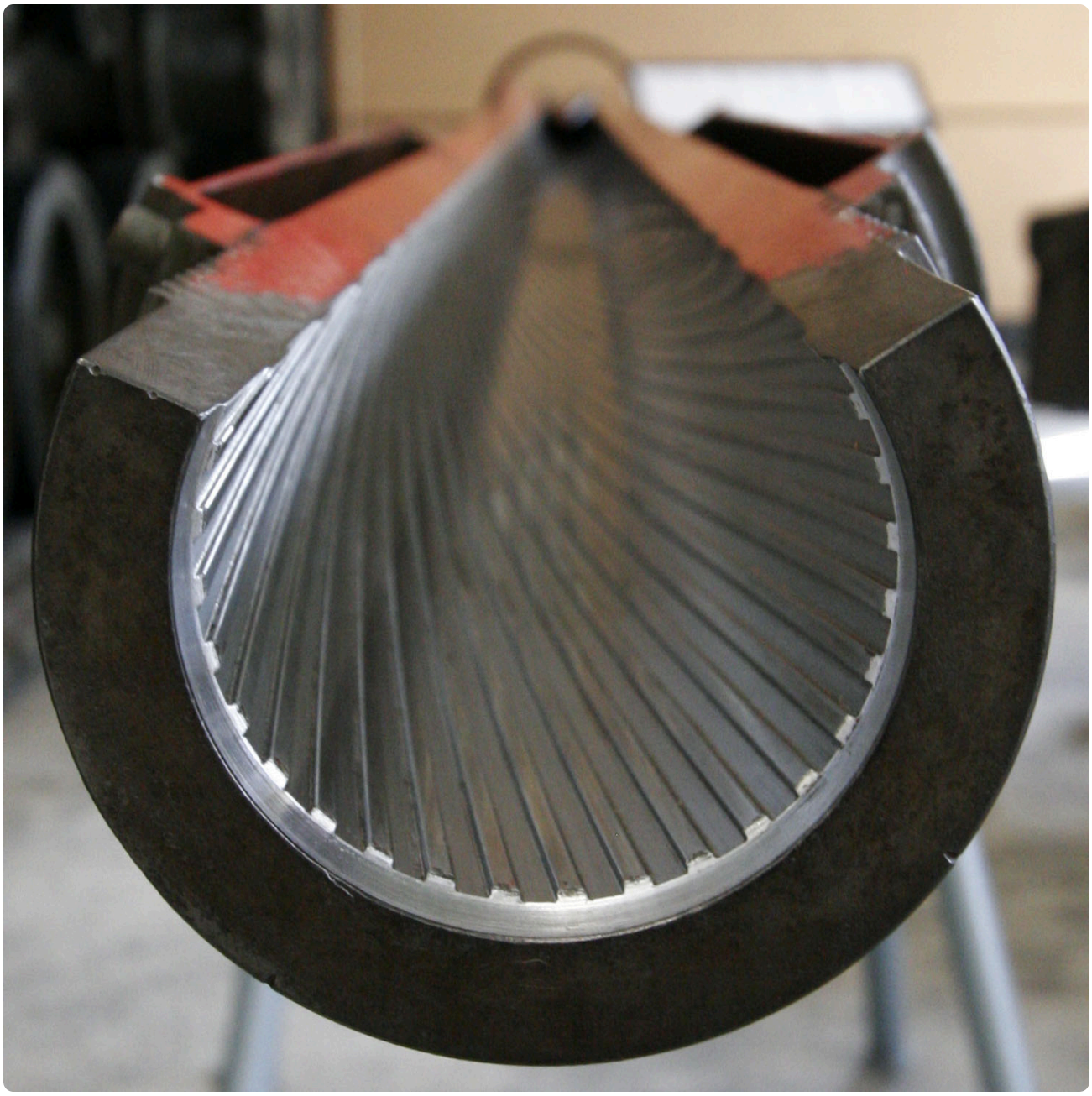
The brazed tube and the milled slot were both manufacturing answers; the milled slot turned out to be the better physics. A rectangular channel shares each side wall with its neighbor, and that wall, the *rib*, is a fin: heat conducts up it from the hot wall and into the coolant on both faces. Make the channels tall, narrow and many, and the ribs reach deep into the coolant, adding surface at about the same flow area. Carlile and Quentmeyer measured this at NASA Lewis in 1992 [10]: three copper chambers differing only in channel shape, where at the same coolant pressure drop the chamber with an aspect ratio of 5 ran its hot wall at 539 K against the 765 K of the baseline's 0.75, 30 percent cooler, and showed no fatigue damage after 440 thermal cycles.

Printing adds one more freedom: a channel does not have to run straight. Printed copper chambers wind many square-ish passages around the wall as a helix, so a coolant particle can make several laps of the chamber before it leaves. Held at a constant angle to the wall's meridian, the passages all fit side by side, and the rib between them grows wherever the wall's radius does.

NOVA builds three channel families from these ideas, one per run, set by `channelType`:

CHANNELTYPE	SECTION	HELD	SIZED	FALLS OUT
`circle`	Circle	Rib	Radius	Wrap angle
`rectangular`	Rounded rectangle, depth along the wall normal	Rib, and the channel runs straight	Depth	Width and aspect ratio
`helical`	Rounded rectangle on a helix of N starts	Helix angle and aspect ratio	Width	Rib

Spirally *fluted* channels are kept in `experimental/flutedChannels.py`: a circle with helical grooves, like the rifling in a gun barrel or the ridges of a churro, rolled so the coolant spins and the thermal boundary layer never settles. Their appeal is to a coolant short on capacity, oxygen in a hybrid above all. The geometry builds, but no source supports the heat transfer correlation they were rated with, so they are not one of the families NOVA offers.



Sectioned 105 [mm] tank gun barrel showing the helical lands and grooves, the analogue for a fluted channel. Credit: baku13, CC BY-SA 3.0

Defining the Channel Geometry

The channel geometry is not drawn; it is derived. NOVA solves the channel station by station against the wall temperature it is allowed, and the geometry is whatever falls out of that solve. Everything below follows from the contour, the coolant conditions and a handful of design parameters: the channel family and count, the hot wall and rib thicknesses, and for a helix its angle and aspect ratio.

Where does the jacket run?

The jacket covers the regen section: from the injector face, over the chamber barrel and through the throat, to wherever the regen section is truncated, or to the exit. How the coolant runs over it is the circuit. On a single pass (architecture A) it enters through the *inlet volute* at the aft end and runs forward against the exhaust, so it meets the lowest heat flux first and arrives at the throat already warmed, and leaves through the *outlet volute* at the injector face; `jacketFlowDirection` `coflow` runs it the other way. The multi-pass circuits, the extension's own circuit and the films are in Regenerative Cooling Architectures, which also lists the example engine in `assets/examples/` for each architecture. The shipped engine runs a one-and-a-half pass (B) to the exit.

At each end the channel has to turn off the wall and out to its volute. A plane is drawn normal to the axis, `inletVoluteAxialOffset` upstream of the aft end (or `outletVoluteAxialOffset` downstream of the injector face), and a fillet is placed tangent to both the wall and that plane. The channel follows the wall into the fillet, turns through it, and leaves along a straight flare of `inletVoluteFlareLength`, tilted `inletVoluteTilt` from the radial, where the volute attaches. The fillet's radius is `inletVoluteFlareRoverD` times the jacket's full radial depth at that station: hot wall, the largest channel that fits there, and shell. The outlet end is the same construction reflected through a plane normal to the axis, `upstreamVoluteInterfaceCurve()`, so for the same settings the two ends are exact mirror images.

Sizing the channel

The quantity solved for is the section's *radial half-extent*, `channelRadius`: a circle's radius, or half a rectangle's depth. Measuring every family by how far it reaches from its centerline toward the wall and away from it means the centerline offset, the shell and the volute placement are the same for all three.

At a fixed coolant flow, a smaller channel carries the coolant faster, which cools the wall harder and costs pressure; a larger one costs less pressure and runs the wall hotter. So at each station the answer we want is *the largest channel that holds the hot wall at* `maxWallTemperature`, which is also the channel with the least pressure drop. The sizing march in `solveChannelRadii()` starts at the coolant inlet, proposes a size, rebuilds the section, runs the thermal model at that station, reads the wall temperature back, and adjusts with an adaptive secant search until the two agree to one part in ten thousand. The search relies on the wall temperature rising monotonically with channel size, which the tests hold a rectangle's depth to.

Each family caps the size differently:

- **Circle:** the largest circle that packs between its neighbors with the rib left over, tangent to the wall offset by the hot wall:

$$r = \frac{R\sin(\pi/N)}{1-\sin(\pi/N)} - \frac{t_{infill}}{2}, \qquad R = r_{wall} + t_{wall} - t_{infill}$$

- **Any family:** the depth may not pass `maxChannelDepth` where one is given. A section's depth is twice its half-extent in every family, so a circle's depth is its diameter, and this is the only thing bounding a circle from above apart from the room between its neighbors.
- **Rectangular:** the depth may reach `maxChannelAspectRatio` times the width, and `maxChannelDepth` where one is given.
- **Helical:** the width may reach the pass spacing less the minimum rib, and the depth follows it through `channelAspectRatio`, again within `maxChannelDepth`.

Below, every family is held to the smallest the process can build: `minChannelRadius` for a circle or a rectangle's half-depth, `minChannelWidth` for a helix. Before the march starts, the throat is checked: if the channel that fits there is smaller than the minimum, `nChannel` is reduced until it is not.

The two kinds of bound are not treated alike. A station held at an upper bound leaves its wall cooler than the limit, which is a jacket with margin to spare, so the search accepts it and reports the temperature it came out at. A station held at the lower bound with its wall still over the limit has nothing left to try, because only a smaller channel cools harder, so it stops the run and names the station, the wall it reached, and what would fix it.

Leaving `maxChannelDepth` unset has a consequence worth knowing. Both the objective and the wall temperature constraint improve as a channel grows: a larger channel drops less pressure and the wall only runs hotter, which is free wherever the wall is nowhere near its limit. So where the heat flux is low the search grows the channel until it fills its pitch. On the reference nozzle jacketed to the exit on a single pass, that puts 45 of 60 stations on the packing bound, with the aft channels 33 mm across carrying hydrogen at about 0.9 m/s. The depth limit is what makes the problem bounded from above.

What the coolant is worth when it leaves is a separate question from the wall, and it is asked separately. `minCoolantExitPressure` and `minCoolantExitTemperature` are checked once the jacket is solved, because a jacket can hold every wall at its limit and still arrive at the injector with nothing left to inject with. Each is ignored where the configuration set no limit.

Fixing the size by hand

The search is the honest way to size a jacket and it is not cheap: seven or so passes of the thermal model per station, where marching a known geometry takes one. That is a poor trade when the jacket is already decided and something else is being changed, so `channelSizingMode` offers the other half of the loop. Set it to `manual` and the half-extent comes from `manualChannelProfile` instead, one pass per station, and the wall temperature, the pressure drop and the coolant exit state come out as results. Nothing holds the wall anywhere: the run prints the hottest station and its margin against `maxWallTemperature` and leaves the judgement to the reader.

A profile is a half-extent in meters for a constant channel, or a list of `[key, half-extent]` control points, linear in the key between points and flat at the nearest point outside them. `manualChannelProfileKey`

picks the coordinate: `areaRatio` is the local area ratio signed negative upstream of the throat, so the throat is -1 on the converging side and +1 on the diverging side and nothing sits between them, and `jacketFraction` is the meridional distance from the injector face as a fraction of the whole jacket. The first is the key to write a profile in by hand, anchored to the throat rather than to the length of one particular jacket; the second is the key a recorded profile is written in, because a constant radius barrel holds a single area ratio over its whole length and a profile keyed on area ratio cannot vary along it.

The bounds are the same ones the search respects, and here they are hard. A station whose requested size will not fit between its neighbors, or falls below the smallest the process can build, stops the run and names the station, its axial position, its key, the size asked for and the bound. A channel count too high for its throat stops it too. The geometry would otherwise shrink the channel to what fits, which is the right answer for a search and the wrong one for a profile.

Every run, in either mode, records the profile it built as the configuration entries that replay it: printed at the end of a search, and written to `<name>ChannelProfile.json` beside the exported geometry. A configuration may name that file as its `manualChannelProfile` rather than listing points. Replayed at the station count it was recorded at, it reproduces the jacket it came from bit for bit; at a different `numCrossSections` it carries the distribution rather than the numbers, since the key is a fraction of the jacket.

Wrapping the channel

A circle does not fill its share of the circumference, so it is wrapped around the wall: Kineo's Algorithm (named for Kineo, who wrote it and is the only person who really knows how it works) turns each station through the angle that makes the channel's projection fill its pitch, so there are no gaps between channels.

A rectangle fills its pitch by construction, so it runs straight.

A helix runs at a constant angle ϕ to the meridian of the cold wall, which makes it a *loxodrome*, the path a ship holding one compass heading traces on a globe:

$$\frac{d\theta}{ds_m} = \frac{\tan\phi}{r_{cw}}$$

where θ is the wrap angle, s_m the arc length along the cold wall's meridian, and r_{cw} the cold wall's radius. On a cylinder it winds at a constant rate; on a cone it winds as the logarithm of the radius, tighter where the wall is narrow. The helix holds its azimuth through the volute interfaces, so it winds only where it lies on the jacketed wall.

With the wrap angles in hand, the 2D centerline is rolled about the nozzle axis into three dimensions.

The cross sections

A circle is drawn in the plane normal to the centerline, on a parallel transport frame: a frame that follows the curve without twisting about it. A circle is the same shape at any roll, so that is all it needs.

A rectangle is not, and its depth has to point along the wall normal at every station. It is drawn on frames built from the wall normal instead, `wallNormalFrames()`: the normal of the wall's meridian, rotated to the station's azimuth, with the binormal completing the set. A parallel transport frame would drift off the wall normal wherever the centerline wraps, which is everywhere on a helix.

The rectangle's corners may be rounded at `channelCornerRadius`, and its flow area and wetted perimeter are

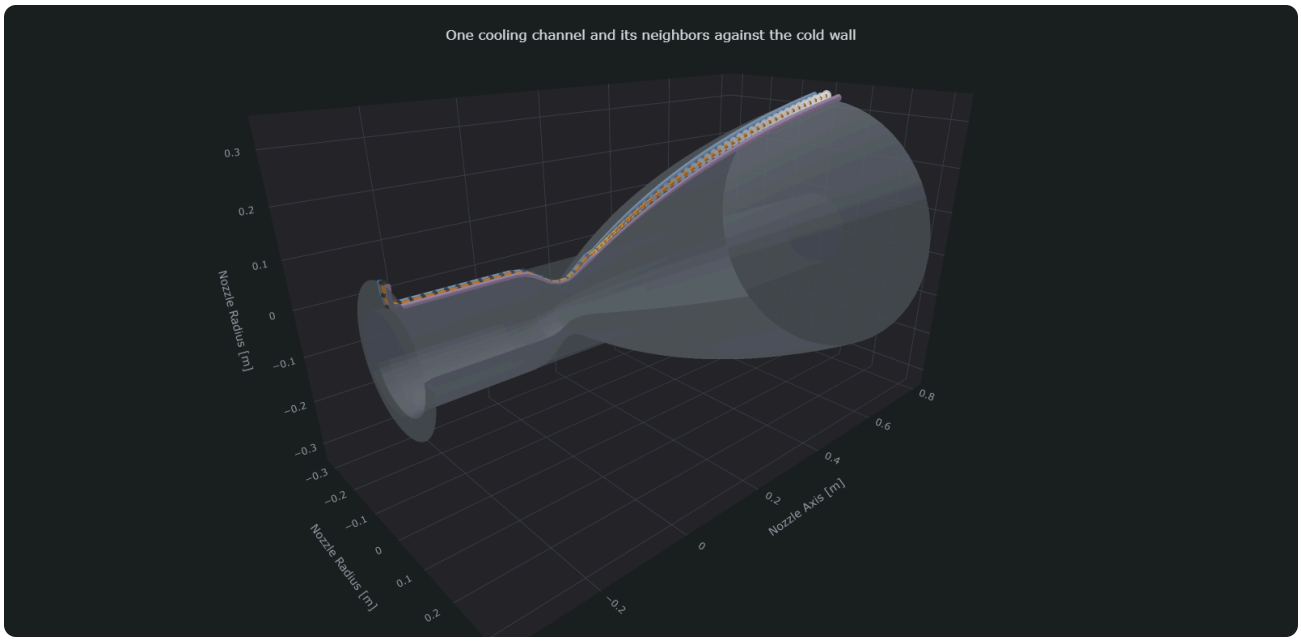
$$A = wd - (4 - \pi)r_c^2, \qquad P = 2(w + d) - (8 - 2\pi)r_c$$

with w the width across the wall, d the depth outward from it, and r_c the corner radius, clamped to half the smaller side. Each family sets w and d its own way:

- **Rectangular:** the width fills the pitch at the cold wall less the rib, $w = 2\pi r_{cw}/N - t_{infill}$, so the rib at its root is exactly `infillThickness`, and the depth $d = 2\cdot$ `channelRadius` is sized.
- **Helical:** the passes sit $s = 2\pi r_{cw}\cos\phi/N$ apart measured across the channel. The width is sized, the depth is `channelAspectRatio` times it, and the rib is what the spacing leaves, $t_{rib} = s - w$, never less than `infillThickness`.

The jacket

With one channel defined, the jacket is that channel patterned about the nozzle axis `nChannel` times.



One up-family channel and its two neighbors swept against the cold wall from the injector face to the exit, with the centerline and every cross-section called out on the middle one. The up family is this channel patterned about the axis `nChannel` *times; aft of the inlet manifold at an area ratio of three the down family runs between them, not drawn here. LOX/LH2 100 [kN] upper stage example case, one-and-a-half pass*

Modeling the performance

To keep design iteration quick, the heat transfer is estimated by a one-dimensional thermal resistance model along the wall. Heat flow plays the part of current and temperature the part of voltage, so at each station

$$\dot{Q} = \frac{T_{aw} - T_{coolant}}{R_{gas} + R_{wall} + R_{coolant}}$$

with $\dot{Q}$ the heat through one channel's share of the wall.

The gas side is the Bartz correlation [8], with the boundary layer correction σ carrying the whole dependence on the wall temperature:

$$h_g = \left(\frac{0.026}{D_t^{0.2}}\right)\left(\frac{\mu^{0.2} c_P}{Pr^{0.6}}\right)\left(\frac{P_c}{c^*}\right)^{0.8}\left(\frac{D_t}{R_c}\right)^{0.1}\left(\frac{A_t}{A}\right)^{0.9}\sigma$$

$$\sigma = \frac{1}{\left(\frac{1}{2}\frac{T_{wall}}{T_c}\left(1+\frac{\gamma-1}{2}M^2\right)+\frac{1}{2}\right)^{0.68}\left(1+\frac{\gamma-1}{2}M^2\right)^{0.12}}$$

with R_c the throat's radius of curvature, the stagnation Prandtl number $4\gamma/(9\gamma-5)$ and the stagnation viscosity $1.184\times10^{-7}MW^{0.5}T_0^{0.6}$. Because σ depends on the unknown wall temperature, each station is converged. The gas-side area is each channel's share of the wall, $A_{hw}=(2\pi r/N)\,ds_m$ over the wall's own meridional length, so the channels tile the wall exactly at any wrap angle.

The wall conducts through the sector of the cylindrical shell each channel owns:

$$R_{wall}=\frac{r\ln(1+t/r)}{kA_{hw}}$$

which tends to the slab $t/(kA_{hw})$ for a wall thin against its radius.

The coolant side uses Gnielinski's Nusselt number [9] on the hydraulic diameter $d_H=4A/P$ (a circle's $2r$), with the Swamee-Jain friction factor [11]:

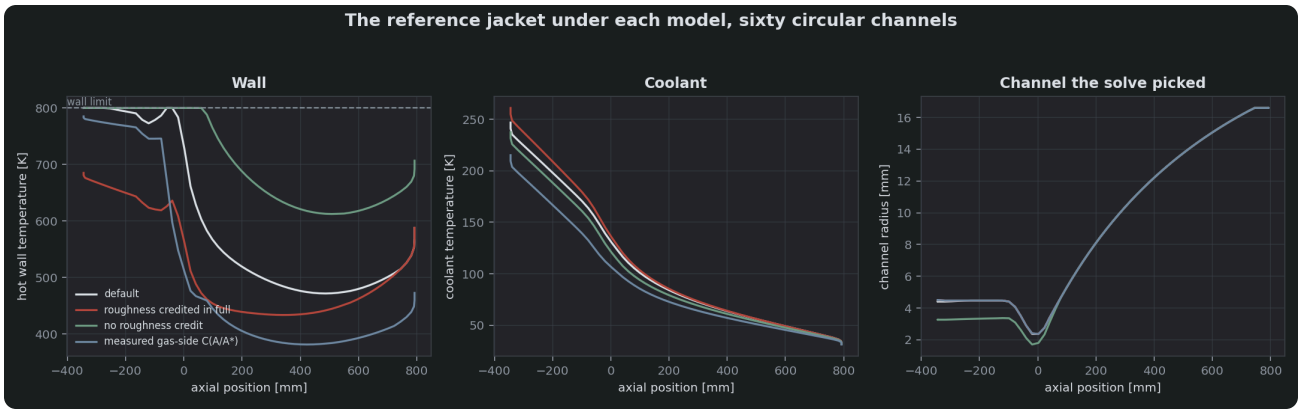
$$Nu=\frac{(f/8)(Re-1000)Pr}{1+12.7(f/8)^{1/2}(Pr^{2/3}-1)},\qquad f=\frac{0.25}{\log_{10}^2\left(\frac{e}{3.7d_H}+\frac{5.74}{Re^{0.9}}\right)}$$

and $h_c=k_{fluid}Nu/d_H$, with the coolant properties at the local state from REFPROP or CoolProp. A circle takes its heat through the half of its perimeter facing the wall. A rectangle or a helix takes it through its floor and through its two side walls as the faces of the ribs, each a straight fin of height H and thickness t_{rib} cooled on both faces with an adiabatic tip:

$$A_{coolant}=(w_{floor}+2\eta H)ds,\qquad \eta=\frac{\tanh(mH)}{mH},\qquad m=\sqrt{\frac{2h_c}{k\,t_{rib}}}$$

The coolant then carries the heat it picked up to the next station, added to its enthalpy rather than taken as a rise at the station's own c_p, which conserves the heat the wall gave up even where c_p swings across the station. It loses pressure to friction and to the bends in its path.

The heat transfer model outputs plots of the coolant properties, both heat transfer coefficients and the hot wall temperature along the nozzle axis. These are a first pass: a design worth building should still go through three-dimensional conduction and CFD.



The reference jacket solved under each selectable model: hot wall temperature against its limit, coolant temperature, and the channel size the sizing march picked. Sixty circular channels, LOX/LH2 100 [kN] upper stage example case

How far can we trust it?

The gas side is checked against a worked example of the correlation as NASA SP-125 gives it, but not against a measurement of its own. Measurements exist, and they do not flatter it. Written as $St^* Pr^{*0.7} = C\, Re_d^{*-0.2}$ with properties at Eckert's reference enthalpy, the Bartz form carries $C = 0.026$ along the whole wall, while Schacht, Quentmeyer and Jones measured C at five stations of a LOX/GH2 chamber over 150 to 1000 psia [13] and found it varies: 0.0257 in the cylindrical barrel, 0.0240 through the converging section, 0.0148 at the throat, and 0.0153 to 0.0188 downstream. Four other configurations put the throat between 0.017 and 0.023. So the coefficient here is about right over the barrel and 20 to 40 percent high at the throat, where the flux is highest. A high coefficient is not a licence: it drives the sizing march to a smaller channel, which holds the wall below its limit and charges pressure drop for margin that was already there.

Setting `gasSideAxialModel` to `measured` scales the coefficient by that measured distribution, normalized to the barrel so the absolute level stays Bartz's and only the shape comes from the measurement. On the full-contour regression jacket it leaves the barrel alone, takes 41 percent off the throat, and costs 15 K of coolant pickup for 12 percent less pressure drop. It is a calibration from one engine and one injector rather than a correlation, and the source says so itself; the account is in gasSideAxialDistribution_2026-09-22.md.

Setting `gasSideAxialModel` to `ievlev` runs levlev's integral method as RPA carries it, in `ievlevHeatTransfer`. It integrates the energy equation along the wall in closed form, so a station's coefficient depends on the wall upstream of it, and the jacket is sized around an outer iteration on the wall temperature that closes in two or three passes. On the full-contour regression jacket it carries more heat into the coolant than Bartz, 266.5 K at the outlet against 246.9 K, with a wider throat channel. Against the measured heat flux of Test 024 on the MSFC 40k calorimeter chamber, on the test's own wall

temperature, it runs the throat 20 percent high against the barrel, where Bartz runs it 38 percent high; the comparison is in calorimeter40k_2026-10-04.md.

The coolant side was put against Carlile and Quentmeyer's three chambers [10], at one station with the gas side fixed from their stated operating point. Every one of their 13 measured throat wall temperatures lies inside the band NOVA predicts once the coolant state and the channel roughness, neither of which the paper reports, are bracketed. The bands are wider than the tolerances set before the comparison, so it is a comparison, not a validation.

What a rough wall is allowed to do to the heat transfer is the least settled part of the coolant side, and `coolantRoughnessModel` makes the choice explicit. Roughness raises the friction factor, and the pressure drop with it, under all three options. The default credits it with the rough-wall heat transfer Dipprey and Sabersky measured [12], which rises with roughness by less than the friction does. Their expression is held between the smooth-wall and full-credit results, because outside its fit (small roughness Reynolds numbers, and the Prandtl numbers of 20 to 30 of a liquid hydrocarbon) it leaves both: it does not reduce to the smooth-tube result as the roughness vanishes, and at Pr 25 it reads three times the smooth wall where the wall is barely rough. Held there, it is continuous in the roughness Reynolds number, a smooth wall is exactly Gnielinski, and hydrogen on a printed wall is unchanged because it sits inside both bounds. Putting the rough friction factor straight into Gnielinski's smooth-tube form instead credits heat transfer in proportion to friction, which no measurement supports, and taking the heat transfer on the smooth-wall factor credits it with nothing, which is what NASA TN D-7207 did [14].

The two hardware comparisons bracket the answer rather than agreeing on it. Carlile and Quentmeyer's wall temperatures want more coolant-side heat transfer than a smooth wall gives: crediting nothing puts 2 of their 13 measurements below the predicted band. The hydrogen coefficients of TN D-7207 want less: against those, crediting nothing lands nearest and the default runs 1.4 to 1.8 times high. A coefficient that is too high cools the wall, so the two datasets pull in opposite directions, and the default sits between them. The accounts are in carlileQuentmeyer_2026-09-22.md and coolantSideAgainstTND7207_2026-09-22.md.

Three more things the model does not see, and it is the onus of the designer to allow for them:

- Over the chamber barrel, Bartz is used outside the throat region it is referenced to. On a straight barrel its coefficient varies only through the wall temperature, with no injector near field and no boundary layer start, and the barrel's gas state takes the chamber pressure as stagnation with no Rayleigh loss.
- The coolant is taken as fully mixed across a tall channel, where in reality it stratifies from floor to roof.
- A helical passage curves around the chamber with the hot wall on the inside of the bend, where secondary flow is expected to reduce the heat transfer, so the model is optimistic for a helix, more so the steeper its angle.

Part III: Worked Design Example

To tie both parts together, this section carries a single design point end to end: a 100 [kN] LOX/LH2 upper stage nozzle. It is a useful example because the operating point is close to peak specific impulse for the propellant combination and the chamber thermochemistry can be checked directly against the NASA CEARun web tool.

Design Point

PARAMETER	VALUE	NOTES
Propellants	LOX / LH2	
Chamber pressure	6.8948 [MPa]	1000 [psia]
Mixture ratio	5.5	hydrogen-rich, near peak I_{sp}
Expansion ratio	40	geometric, where the contour is cut
Thrust	100 [kN]	mass flow follows from it
Oxidizer / fuel inlet temperature	90.17 [K] / 20.27 [K]	normal boiling points
Contour	truncated ideal, $L_{frac} = 0.8$	method of characteristics
Chamber diameter	180 [mm]	contraction ratio approximately 3.2

Chamber Conditions

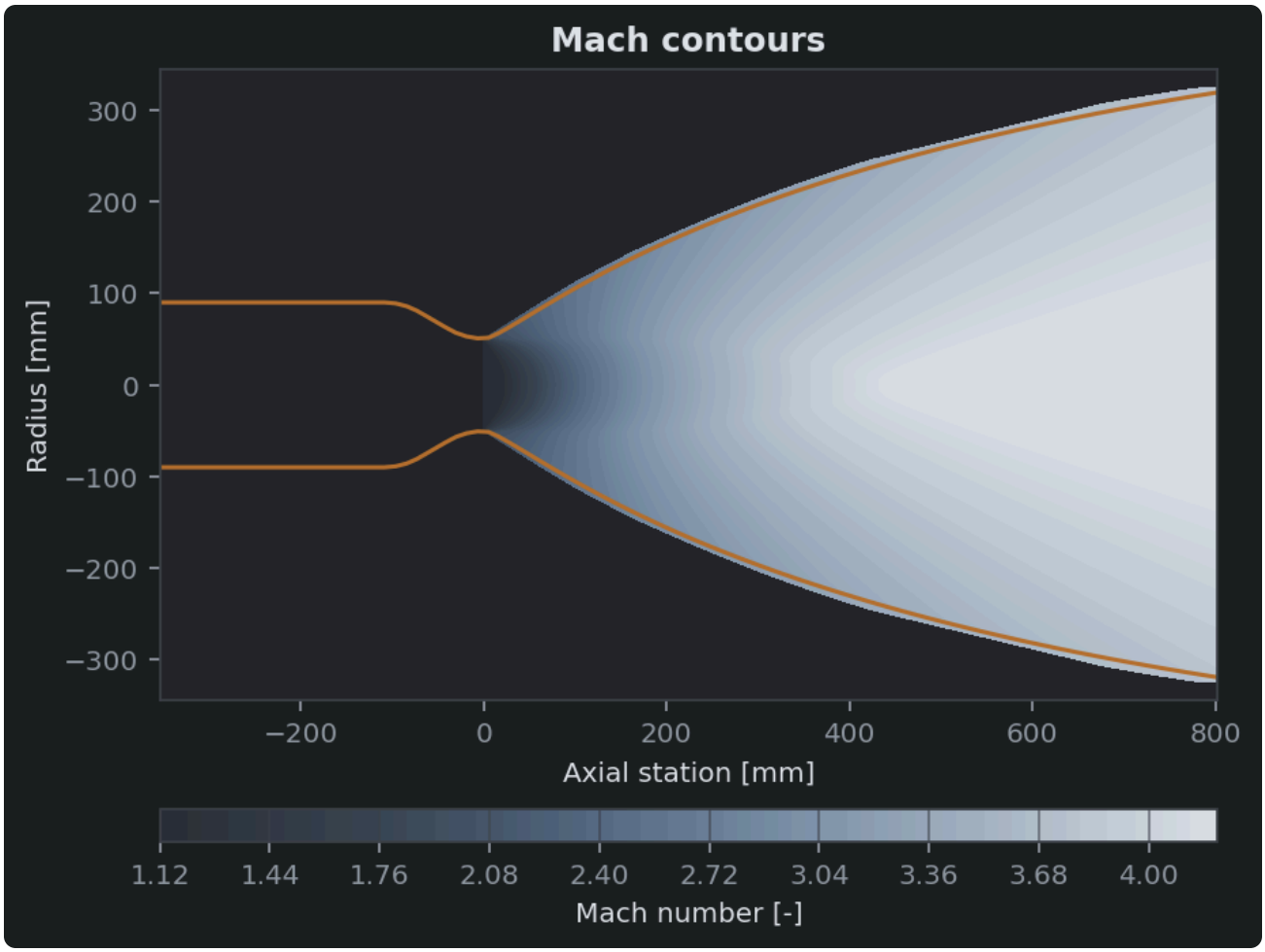
Equilibrium chamber properties, compared against CEARun for the same case:

QUANTITY	CALCULATED	CEARUN
Chamber temperature	3398.4 [K]	~3400 [K]
Chamber molecular weight	12.662 [g/mol]	~12.7 [g/mol]
Chamber γ	1.1475	~1.14
Characteristic velocity c^*	2339.9 [m/s]	~2330 [m/s]
Vacuum I_{sp}	453.77 [s]	~450 [s]
Chamber Prandtl number	0.5190	~0.5

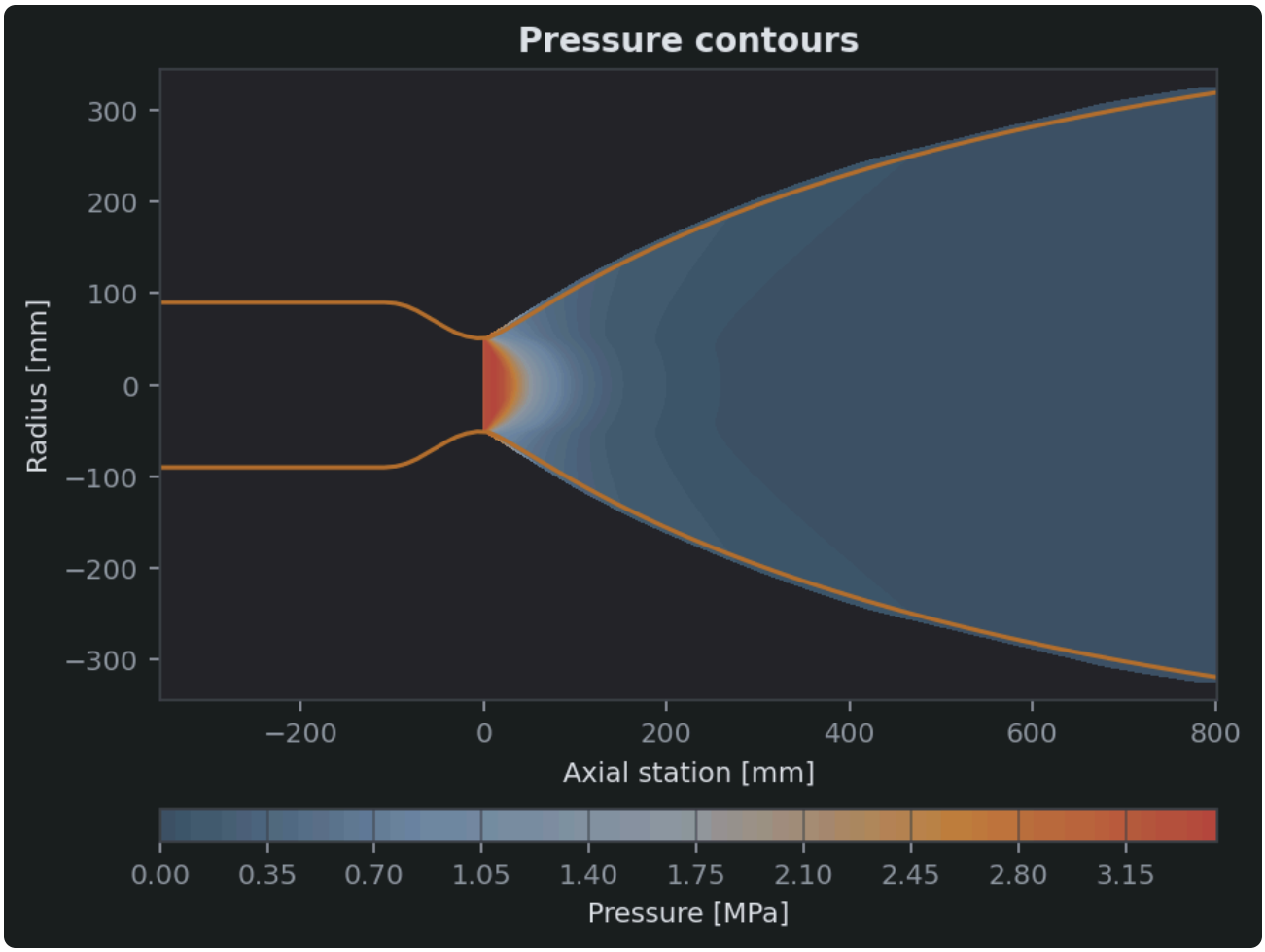
Resulting Geometry and Performance

QUANTITY	VALUE
Throat radius	50.5 [mm]
Exit radius	319.2 [mm]
Diverging section length	802.4 [mm]
Mass flow	23.466 [kg/s]
Ideal I_{sp}	434.4 [s]
Thrust coefficient, vacuum, delivered	1.907, 1.876 after wall friction (1.945 one-dimensional in the chamber gas)
Thrust coefficient at the design back pressure of 13.99 [kPa], delivered	1.826, 1.795 after wall friction
Specific impulse, vacuum, delivered	461.5 [s], 453.9 [s] after wall friction

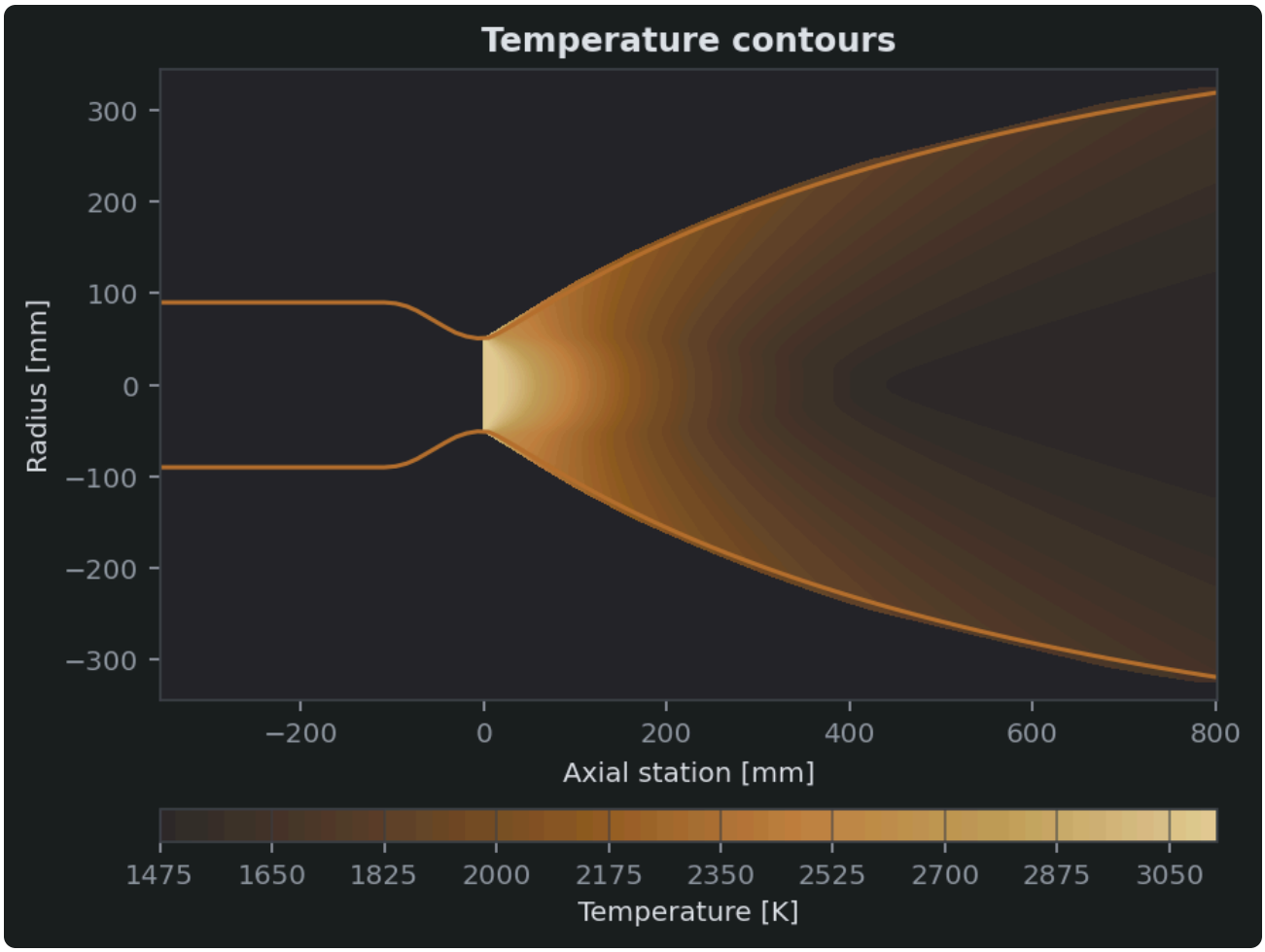
The characteristic mesh runs sonic at the throat. At the exit plane it runs from Mach 3.78 at the wall to 4.19 on the axis.



Mach number contours over the generated Mach net



Static pressure contours over the same mesh



Static temperature contours over the same mesh

A Note on Area Ratio

The configured expansion ratio of 40 is the **geometric** area ratio of the delivered nozzle. The solve finds the ideal contour that reaches that area ratio at 80 percent of the length of a 15 degree cone and cuts it there, so the exit pressure is a result of the cut rather than a target.

A truncated contour has no single exit pressure. The one-dimensional exit state at an area ratio of 40 is 13 993 [Pa] in CEA's equilibrium gas and 17 699 [Pa] in the frozen chamber gas the characteristics are solved in. Across the method of characteristics exit plane the static pressure runs from 25.5 [kPa] at the wall to 10.7 [kPa] on the axis, with a mass average of 18.6 [kPa]. The wall carries the highest pressure because the cut stops the bell before its wall has expanded to the core's Mach number.

References

[1] Cronvich - A Numerical-Graphical Method of Characteristics for Axially Symmetric, Isentropic

[2] Sauer - General Charcateristics of the Flow Through Nozzles at Near Critical Speeds

[3] Young - Automated Nozzle Design through Axis-Symmetric Method of Characteristics Coupled with Chemical

[4] Yu - A Summary of Design Techniques for Axisymmetric Hypersonic Wind Tunnels

[5] GRCop-42 and -84 Typical Average Summary

[6] Webb, Kim - Principles of Enhanced Heat Transfer

[7] RPE Nozzle Design

[8] Bartz - A Simple Equation for the Rapid Estimation of Rocket Nozzle Convective Heat Transfer

[9] Gnielinski - Neue Gleichungen fur den Warme- und den Stoffubergang in turbulent durchstromten Rohren und Kanalen (New equations for heat and mass transfer in turbulent flow pipes and channels)

[10] Carlile, Quentmeyer - An Experimental Investigation of High-Aspect-Ratio Cooling Passages, NASA TM-105679, 1992

[11] Swamee, Jain - Explicit Equations for Pipe-Flow Problems, Journal of the Hydraulics Division, ASCE, 1976

[12] Dipprey, Sabersky - Heat and Momentum Transfer in Smooth and Rough Tubes at Various Prandtl Numbers, International Journal of Heat and Mass Transfer, 1963

[13] Schacht, Quentmeyer, Jones - Experimental Investigation of Hot-Gas Side Heat-Transfer Rates for a Hydrogen-Oxygen Rocket, NASA TN D-2832, 1965

[14] Schacht, Quentmeyer - Coolant-Side Heat-Transfer Rates for a Hydrogen-Oxygen Rocket and a New Technique for Data Correlation, NASA TN D-7207, 1973

Sources for the channel families and the hardware comparison are annotated in references_regenChannels_2026-09-22.md.

Reference photography and diagrams carry their own credits and licenses in referenceImages/imageCredits.md.

Authors

Part I, Nozzle Contour Design: Sean Bowman, last updated 03/21/2025

Part II, Regenerative Cooling Architecture Design: Isabella Duprey-Churn, last updated 07/16/2025

Consolidated overview compiled 07/20/2026